

Slovak University of Technology
Faculty of Informatics and Information Technologies

FIIT-182905-116308

Bc. Matej Ševčík

E-Learning SW Modeling Using a Class Diagram

Master's Thesis

Supervisor: Ing. Vladimír Mlynarovič, PhD.

May 2026

Slovak University of Technology
Faculty of Informatics and Information Technologies

FIIT-182905-116308

Bc. Matej Ševčík

E-Learning SW Modeling Using a Class Diagram

Master's Thesis

Study programme:	Intelligent Software Systems
Study field:	Informatics
Training workplace:	Institute of Informatics, Information Systems and Software Engineering
Supervisor:	Ing. Vladimír Mlynarovič, PhD.
May 2026	

I declare in my honor that I have prepared this work independently, on the basis of consultations and using the mentioned literature.

In Bratislava, May 2026

Bc. Matej Ševčík

Annotation

Slovak University of Technology Bratislava
Faculty of Informatics and Information Technologies
Degree Course: Intelligent Software Systems

Author: Bc. Matej Ševčík
Diploma Thesis: E-Learning SW Modeling Using a Class Diagram
Supervisor: Ing. Vladimír Mlynarovič, PhD.
May 2026

E-learning is a perspective field that offers many opportunities to support education. It has the potential to increase the effectiveness of learning, reduce the burden on educators, make education more accessible to the general public, and introduce innovative practices that traditional educational approaches do not allow. Despite this, e-learning is used relatively rarely in Slovak schools, even at faculties of informatics. In this project, we focused on the possibilities of applying e-learning to teaching the UML class diagram, with the potential for expansion to other UML diagrams. We analyzed various techniques used in e-learning and their application in teaching software modeling. The main goal of this project was to create a course for teaching the UML class diagrams. Based on our analysis, we selected several e-learning practices and implemented them in the course using two different content designs - competitive and playful. We conducted an experiment on students and compared the effects of the designs on their learning performance, behavior patterns, and subjective perceptions. Based on the experiment's results, neither approach can be directly linked to better objective performance. Playful design lead to higher engagement and satisfaction. Competitive design shows signs of having a polarizing effect on students' perceptions. Overall, students expressed a positive attitude towards the use of e-learning at Faculty of Informatics and Information Technologies STU in Bratislava (FIIT STU).

Keywords: e-learning, class diagram, software modeling

Anotácia

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
Študijný program: Intelligent Software Systems

Autor: Bc. Matej Ševčík

Diplomová práca: E-Learning modelovania SW pomocou diagramu tried

Vedúci diplomového projektu: Ing. Vladimír Mlynarovič, PhD.

May 2026

E-learning je rýchlo sa rozvíjajúca oblasť poskytujúca veľa možností podpory vzdelávania. Má potenciál zvýšiť efektivitu vzdelávania, znížiť záťaž na pedagógov, sprístupniť vzdelávanie širšej verejnosti alebo zaviesť inovatívne praktiky, ktoré tradičné vzdelávacie prístupy neumožňujú. Napriek tomu je na slovenských školách e-learning využívaný pomerne zriedkavo, a to aj na fakulte informatiky. V projekte sme sa zamerali na možnosti aplikovania e-learningu pri vyučovaní UML diagramu tried s možnosťou rozšírenia na ďalšie UML diagramy. Vykonali sme analýzu rôznych prístupov používaných v e-learningu a ich využitia pri vyučovaní modelovania softvéru. Hlavným cieľom tohto projektu bolo vytvorenie kurzu na výučbu UML diagramu tried. Na základe analýzy sme vybrali niekoľko e-learningových praktík a využili sme ich pri implementácii 2 verzií kurzu - kompetitívnej a zábavnej. Vykonali sme experiment na študentoch, v ktorom sme porovnali vplyv týchto dizajnov na ich objektívne výsledky, správanie a subjektívne vnímanie. Výsledky nepreukázali priamy súvis medzi objektívnymi výsledkami a verziou kurzu. Zábavná verzia sa spájala s vyššou mierou zapojenia a vyššou všeobecnou spokojnosťou, zatiaľ čo kompetitívna verzia preukázala známky polarizácie vo vnímaní študentov. Celkovo študenti vyjadrili pozitívny prístup voči ďalšiemu využitiu e-learningu na FIIT STU.

Kľúčové slová: e-learning, diagram tried, modelovanie softvéru

Návrh zadania diplomovej práce

Finálna verzia do diplomovej práce¹

Študent:

Meno, priezvisko, tituly: Matej Ševčík, Bc.
Študijný program: Inteligentné softvérové systémy
Kontakt: xsevcikm3@stuba.sk

Výskumník:

Meno, priezvisko, tituly: Vladimír Mlynarovič, Ing. PhD.

Projekt:

Názov: E-Learning modelovania SW pomocou diagramu tried
Názov v angličtine: E-Learning SW modeling using a class diagram
Miesto vypracovania: Ústav informatiky, informačných systémov a softvérového inžinierstva, FIIT STU
Oblasť problematiky: e-learning, diagram tried, modelovanie softvéru

Text návrhu zadania²

Diagram tried je najpoužívanejším UML diagramom. Slúži na grafické zobrazenie štruktúry systému pomocou tried, ich atribútov, metód a vzťahov medzi nimi. Tento diagram je významným nástrojom pri analýze, návrhu a dokumentácii softvérových systémov. E-learning je rýchlo sa rozvíjajúca oblasť poskytujúca veľa možností podpory vzdelávania. Má potenciál zvýšiť efektivitu vzdelávania, znížiť záťaž na pedagógov alebo sprístupniť vzdelávanie širšej verejnosti. V štúdiách [1][2] študenti, ktorí sa učili modelovať diagram tried pomocou navrhovaného e-learning modulu, dosiahli v cvičeniach a neskôr aj v testoch lepšie výsledky než skupina, ktorá strávila rovnaký čas štúdiom diagramu tried z iných zdrojov. To demonštruje potenciál e-learningu zvýšiť kvalitu výučby v tejto oblasti. Výskumným problémom je teda optimalizácia výučby modelovania softvéru s využitím e-learningových prístupov, pričom hlavnou otázkou je, ktoré metódy sú najefektívnejšie a ako ich vhodne kombinovať.

Analyzujte existujúce e-learningové prístupy vo vzdelávaní softvérového inžinierstva so zameraním na výučbu UML diagramov. Navrhňte vzdelávací nástroj, ktorý implementuje poznatky z tejto analýzy a sprístupnite ho v rámci prototypu novej e-learningovej platformy, prípadne ako modul do už existujúcej platformy. Otestujte efektivitu nástroja na študentoch bakalárskeho štúdia na FIIT STU pomocou testov. Porovnajte výsledky študentov, ktorí využili na prípravu nami navrhnutý nástroj a tých, ktorí sa učili z iných zdrojov. Prínos projektu bude spočívať najmä v tom, že náš nástroj, analýza a testy pripraví priestor na ďalšie rozširovanie a vylepšovanie možností e-learningu v oblasti modelovania softvéru.

¹ Vytlačiť obojstranne na jeden list papiera

² 150-200 slov (1200-1700 znakov), ktoré opisujú výskumný problém v kontexte súčasného stavu vrátane motivácie a smerov riešenia

Acknowledgements

I would like to thank Ing. Vladimír Mlynarovič, PhD. for supervising this work. Furthermore, I want to express gratitude to Ing. Juraj Vincúr, PhD. for providing the domain and granting access to the faculty servers used to deploy the website developed in this project.

Contents

1	Introduction	1
2	The Class Diagram	5
2.1	Overview of UML	5
2.2	UML modeling tools	7
2.3	Class Diagram	8
2.3.1	Components of Class Diagrams	8
2.3.2	Relationships	10
2.3.3	Example of a Class Diagram	11
3	E-learning and the UML Class Diagram	13
3.1	Introduction to E-learning	13
3.2	Comparison of E-learning and traditional learning	14
3.3	Impacts of various e-learning techniques on students	15
3.4	Applying E-learning to UML Class Diagrams	16
3.5	Learning Management Systems (LMS)	16
3.5.1	Gamification	19
3.5.2	Microlearning	20
3.5.3	Social learning	21
3.6	Common issues in Learning UML	21
4	Analysis summary	23
4.1	Related work	23
4.2	Summary	27
5	Objectives and methodology	31
5.1	Objectives	31
5.2	Methodology	32
6	Design	35

6.1	Solution description	35
6.1.1	E-learning platform	35
6.1.2	Content Structure	36
6.1.3	Content - Theory	38
6.1.4	Content - Exercises	38
6.1.5	E-learning techniques	39
6.1.6	Evaluation	40
6.2	Requirements	41
7	Implementation	45
7.1	Course content	45
7.1.1	Theoretical content	46
7.1.2	Exercises	48
7.1.3	Testing students' knowledge	50
7.1.4	Statistical reports	50
7.1.5	Feedback questionnaire	52
7.2	E-learning techniques	53
7.2.1	Playful course version	55
7.2.2	Competitive course version	57
7.3	Deviations from design	59
7.4	Deployment	59
8	Experiment and Evaluation	63
8.1	Experiment	63
8.2	Data collection	63
8.3	Data preparation	66
8.4	Performance analysis	68
8.5	Behavior analysis	70
8.6	Engagement analysis	71
8.6.1	Textual responses	71
8.6.2	Numeric responses	73
8.7	Prospects for future e-learning projects on FIIT STU	77
8.8	Limitations	78
8.9	Summary	79
9	Conclusion	83
9.1	Summary	83
9.2	Future Work	84
	Resumé	86

List of Abbreviations	96
List of Figures	100
List of Tables	101
Declaration about the use of Generative Artificial Intelligence	107
A Description of Digital Submission	B-1
B Technical Documentation	B-3
B.1 Installation Manual for Local Development	B-3
B.2 Installation Manual for Production Environment	B-5
B.3 Plugin Installation	B-5
C Work Schedule	B-1
C.1 Work Schedule for DP1	B-1
C.2 Work Schedule for DP2	B-2
C.3 Work Schedule for DP3	B-3

Chapter 1

Introduction

Unified Modeling Language (UML) is a widely used tool in software modeling. It describes systems in an abstract way and from various perspectives, which makes the ability to understand it and use it a valuable asset for any professional in the field of information technologies.

Most developers claim to possess knowledge of UML. However, in many cases, their knowledge of the topic is rather limited or even inaccurate¹. This observation highlights the need for improving the approach to teaching of software modeling and UML in educational institutions.

Traditional classes often do not provide enough time and space for an actual teacher to give the necessary amount of attention to each of the students. This often results in late or insufficient feedback on modeling exercises, which may negatively affect the process of learning UML [1].

The answer to these problems might lie in electronic learning, or e-learning. It is a practice in which learning material is delivered through digital technologies.

E-learning can offer students interactive materials with automated immediate personalized feedback, structured and easily accessible learning materials, study and exercise recommendations, while simultaneously allowing teachers to spend more time consulting students instead of explaining the basic concepts to them.

E-learning also provides possibilities for increasing students' engagement in the study materials by introducing gamification elements or a sense of competition to the materials.

¹Donald Bell, "An Introduction to the Unified Modeling Language," *IBM Developer*, <https://developer.ibm.com/articles/an-introduction-to-uml/>, accessed February 23, 2025.

It is common for students to have trouble using the UML class diagram properly. However, it is one of the most important diagram types and is relatively easy to understand, since it clearly links to actual code. For these reasons, we decided to aim this study at designing an e-learning course that would act as complementary material to a software modeling university subject.

Numerous studies demonstrate that e-learning courses and similar tools can improve students' learning performance and engagement in software modeling [2][3][4]. They are also well-received by students. The educational tools and materials in the studies were enriched by common e-learning techniques such as microlearning or gamification. Despite being well-established, these techniques are seldom compared within the same settings in the context of software modeling. It is not clear what effect on learning outcomes different design strategies can have.

Our course will seek to not only utilize the expected positive outcomes of e-learning, but also to compare the effects of different course design strategies on the students' learning outcomes, behavior, and engagement. We will implement a number of e-learning techniques and adapt them into two course variants - playful and competitive.

The two variants will be tested on a sample of students at FIIT STU. The results will then be analyzed and studied approaches compared to determine the effectiveness and further potential of such courses and e-learning techniques in the process of education of software modeling.

Chapter 2

The Class Diagram

2.1 Overview of UML

Introduction

The UML is a standardized visual modeling language. Its first version was published in 1997 and later updated to UML 2.0 in 2005 as an attempt to unify various approaches to modeling software systems¹. The latest version of UML is UML 2.5.1 and it was published in December 2017².

The purpose of the language is to specify, document, and visualize parts of software systems. That is accomplished by designing standardized diagrams, each intended to describe a different aspect of the system, usually from various stakeholders' perspectives. Those may include the architect, programmer, project manager, customer, and others.

UML models contain models of business processes, interactions and relationships between individual components or the structure and functions of components. UML documents all stages of the development of a system, not only the initial part. It is mainly aimed at, but not limited to, object-oriented systems.

A model is a representation of something - in this case, a software system. A model captures the system from a certain viewpoint and omits its aspects that are not relevant to the viewpoint [5].

¹Visual Paradigm, "What is Unified Modeling Language (UML)?," *Visual Paradigm*, <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/>, accessed February 23, 2025.

²Conrad Bock et al., "Unified Modeling Language," *Object Management Group (OMG)*, <https://www.omg.org/spec/UML/>, accessed February 23, 2025.

UML diagrams

UML defines a large number of elements that build up diagrams. There are in total 13 different diagrams defined. Those diagrams represent various aspects and viewpoints of the system. In some sources, 14 diagrams are mentioned. The profile diagram is usually not considered to be a separate diagram type however.

Some diagrams show the structure of the system. Those are called structural diagrams. They do not display the behavior of the modeled elements [6]. They describe the system using objects, attributes, operations and relationships.

The other type of diagrams are behavioral diagrams. They describe how the objects interact and change during runtime. They use sequence, activity, collaboration, and state to represent these changes³.

The structural UML diagrams are:

- object diagram
- package diagram
- class diagram
- component diagram
- composition structure diagram
- deployment diagram

The behavioral diagrams are:

- state machine diagram
- use case diagram
- activity diagram
- interaction overview diagram
- sequence diagram
- communication diagram
- timing diagram

³Visual Paradigm, "What is Unified Modeling Language (UML)? (Behavior vs Structural Diagram)," *Visual Paradigm*, <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/behavior-vs-structural-diagram/>, accessed April 13, 2025.

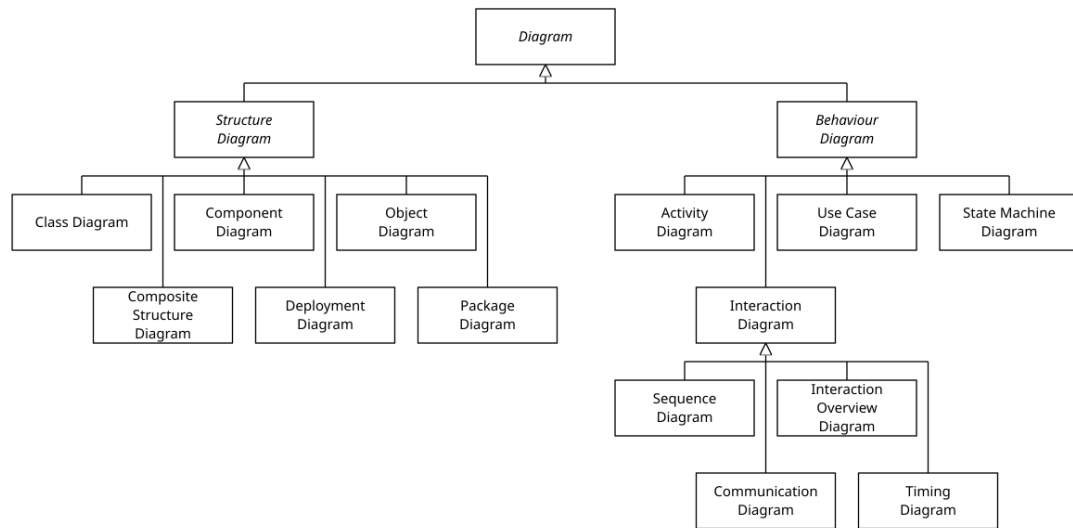


Figure 2.1: Hierarchy of UML diagrams

Source: Dave A. Ryan, "UML diagram," *Wikimedia Commons*, https://en.m.wikipedia.org/wiki/File:Uml_diagram.svg, Accessed: May 23, 2025.

2.2 UML modeling tools

There are many tools for creating UML diagrams available. They differ in complexity, support of features, price or intended use. We selected some of the most popular ones for closer inspection and compared them.

Enterprise Architect⁴

Enterprise is a complex desktop application. It supports a wide range of industry standards, but can have a slow learning curve. It supports many advanced features and has high performance⁵.

Diagrams.net⁶

Diagrams.net, (also known as Draw.io), is a free and simple diagramming tool. It supports UML, but is not specifically aimed at it and does not support some of the advanced functionality of professional modeling tools. It is mainly used by students and for quick sketches.

⁴"Enterprise Architect," *Sparx Systems*, <https://sparxsystems.com/products/ea/>, accessed May 24, 2025.

⁵"Medium", *Icepanel*, <https://icepanel.medium.com/top-9-visual-modelling-tools-for-software-architecture-> accessed May 4, 2026

⁶"Diagrams.net (Draw.io)," *Diagrams.net*, <https://www.diagrams.net/>, accessed May 24, 2025.

Visual Paradigm⁷

Visual Paradigm is a desktop application with a simpler free browser version. It is flexible and supports project management and agile development tracking, while also supporting many industry standard types⁵.

Overall comparison

Tool	Enterprise Architect	Visual Paradigm	Diagrams.net
UML Support	Full	Full	Partial (manual)
Platform	Windows Desktop	Web / Desktop	Web / Desktop
License	Paid (Proprietary)	Free + Paid tiers	Free (Open Source)
Collaboration	Yes	Yes	Limited
Code Engineering	Yes	Yes	No
Templates	Rich	Rich	Limited
Best Use Case	Enterprise systems	Academic / Professional	Quick sketches

Table 2.1: Comparison of UML tools: Enterprise Architect, Visual Paradigm, and Diagrams.net

2.3 Class Diagram

The class diagram is the most prominent UML diagram. It describes the elements of the system as well as their relationships, creating the basis for numerous other diagrams and modeling techniques [7]. These aspects are static, meaning that they do not change with time. Despite that, it is used in multiple phases of system development, applying various levels of abstraction. It is even possible to generate code from the class diagram. It is commonly used as a part of documentation [6], as it provides an easy to understand overview of the system.

2.3.1 Components of Class Diagrams

A class diagram consists of multiple components. In broad terms, those components are classes, their attributes and operations, and the relationships between different classes.

Classes and objects are closely related. A class is a blueprint for creating objects - it describes what attributes and operations an object is going to have and how it is going to interact with other entities.

An object is an instance of a class. For example, a class Table is an abstract idea that is used for creating real tables. A table in the local library is an object - a specific instance of a table that follows the specifications described in the class Table.

⁷"Visual Paradigm," *Visual Paradigm*, <https://www.visual-paradigm.com/>, accessed May 24, 2025.

Attributes

Attributes describe the properties of a class. A class defines the list of attributes along with their data types and visibility. The class may also define an attribute's multiplicity - the number of values that are to be assigned to it. The default amount is 1. In the case of an object, the attributes are assigned specific values.

Operations

Operations are activities that instances of a class can perform. They include things like performing calculations or altering a value. A class describes the visibility, input parameters, return type, and the name of the operation.

Data types

Data types define what kind of information can be stored in a variable (or returned by an operation). Examples are integer, character, string, or even an instance of another class.

Visibility

Visibility sets which entities are allowed to view and interact with an attribute or an operation. There are four types of visibility - public, protected, default and private.

Example of a class

The following figure displays an example of a class from a class diagram. It models a simple User entity that has credentials and a birthdate, allowing the system to update the password and email. It demonstrates the key features of a class, such as attributes, operations, types, operation parameters or access modifiers.

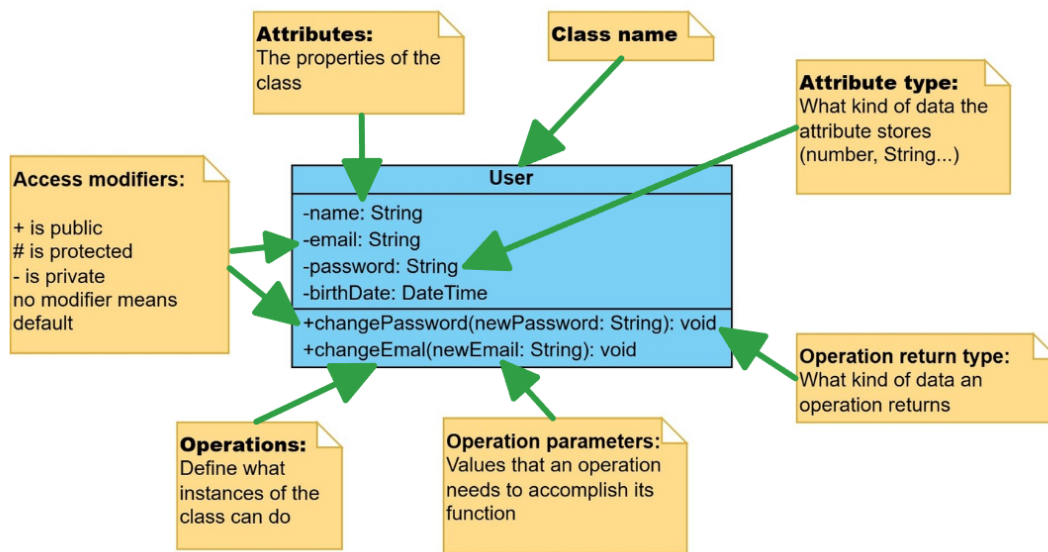


Figure 2.2: An example of a class

Made with: "Visual Paradigm," *Visual Paradigm Online*, <https://online.visual-paradigm.com/>, accessed May 23, 2025.

2.3.2 Relationships

The class diagram defines several types of relationships between different classes, each representing a distinct kind of interaction or connection within a system.

Association

Association represents a general connection between two or more classes. It indicates that objects of one class are connected to objects of another. This is the most basic relationship and can be bidirectional or unidirectional.

Aggregation

Aggregation is a specialized form of association that represents a whole-part relationship between the aggregate (whole) and its parts. Unlike composition, aggregation implies a weaker relationship where the parts can exist independently of the whole.

Composition

Composition is a stronger form of aggregation. It also represents a whole-part relationship, but with the key difference that the part cannot exist independently of the whole. If the whole is destroyed, its parts are typically destroyed as well.

Inheritance

Inheritance (also known as generalization) represents an "is-a" relationship between a more general superclass and a more specific subclass. The subclass inherits the attributes and behaviors of the superclass, allowing for code reuse and polymorphism.

Dependency

Dependency indicates a relationship where one class depends on another because it uses it temporarily, such as by calling its methods or using its data. It is typically a weaker relationship and often represents a "uses-a" connection.

Realization

Realization is a relationship between an interface and a class that implements that interface. It shows that the class provides concrete implementations for the methods defined in the interface.

Cardinality

Cardinality tells how many instances of a class are to be associated with an instance of another class. The most common cardinalities are one-to-one, one-to-many or many-to-many.

2.3.3 Example of a Class Diagram

The following figure displays an example class diagram. It represents a simplified model of an e-shop, where customers place orders and admins manage product catalogs. Orders contain multiple items and can be paid using various payment methods.

It demonstrates many of the essential concepts of a class diagram, mainly inheritance, aggregation, composition, association, multiplicity, realization, and classes with their attributes and operations.

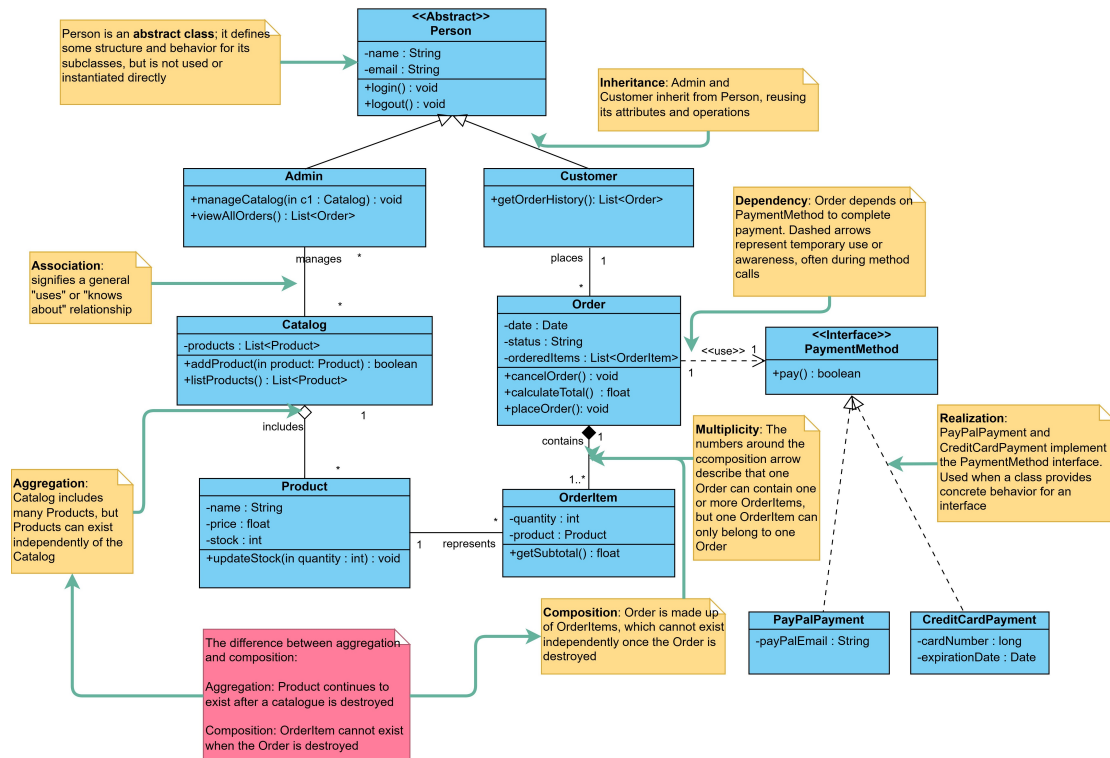


Figure 2.3: An example of a class diagram.

Made with: Visual Paradigm, *Visual Paradigm Online*, <https://online.visual-paradigm.com/>, accessed May 23, 2025.

Chapter 3

E-learning and the UML Class Diagram

3.1 Introduction to E-learning

E-learning, or electronic learning, is a rapidly evolving field that offers many opportunities to enhance education through the use of digital technologies.

It can be used as a complementary tool in traditional classroom settings by introducing various activities such as automatically corrected tests and exercises.

However, digital content can be accessed anytime and from anywhere as long as the learner has a device with an internet connection. That allows e-learning content to be designed for use outside of classrooms without the presence of a teacher.

This is one of its key benefits, since many people do not have access to traditional education due to financial, time or location constraints. It makes education more accessible and self-paced.

Other benefits include the potential to reduce the burden on teachers, increase students' engagement and learning performance by introducing features such as automated instant feedback, personalized content or gamification.

Drawbacks of e-learning commonly lie in reducing the amount of social interactions among learners, varying levels of digital literacy in the population, or lack of access to electronic devices or internet connection in some areas.

3.2 Comparison of E-learning and traditional learning

Traditional learning refers to the model where a teacher or a tutor presents information and exercises to a group of people in a classroom. This model is typical of educational institutions such as universities. This approach has been utilized for a very long time and it is the standard way of teaching.

E-learning offers a number of improvements over the traditional model. Unlike traditional learning, online courses can be taken from anywhere and at any time, removing much of the geographic and time constraints for the learners. It is also significantly cheaper, since it removes the need for a teacher or a learning space. With interactive exercises and materials, e-learning can increase student engagement and interest in the topic.

On the other hand, it also presents a set of challenges that make traditional learning still the preferable choice in most cases. The most obvious drawback is that the self-directed nature of e-learning requires a level of discipline from the learners which many lack. Other problems include a lack of social interactions with other students, a lack of personalized feedback from a live instructor and the fact that some people do not have access to the technology necessary to utilize e-learning materials. For some people, digital literacy may also be a problem stopping them from using e-learning resources.

In a study performed by Ramle et al.[2] a UML course was introduced. Students that took the course were tasked by filling out a questionnaire about their experience. The reception was mostly positive, with students claiming it improved their understanding of UML.

Jurgelaitis et al.[3] found that the introduction of gamified mechanics to a university UML course increased both students' motivation and their average scores. This confirms the general expectations from other works that have not yet collected the necessary data for evaluation.

Lastly, Soler et al.[8] proposed a tool for automated correction of class diagram exercises. The researchers conducted an experiment where students were divided into 2 equally sized groups: one solved a set of exercises using the tool, and the other solved them manually. The number of students who solved all of the exercises correctly was higher in the group that used the tool, and this group also had better average score in a subsequent exam.

These studies clearly demonstrate that tools can be developed to increase not only the students' knowledge, but also their engagement and overall satisfaction with their educational process.

3.3 Impacts of various e-learning techniques on students

There are numerous techniques applied in e-learning materials. Some of them are aimed at driving engagement by introducing game-like aspects to educational content (gamification), some help structure materials better by dividing into small, manageable pieces (microlearning) or by making personalized content recommendations to students (adaptive learning). Social learning strives to leverage positive effects of group activities despite the digital setting.

These techniques can be implemented in numerous ways, and they do not have the same effect on the learners. Using too many is likely to overwhelm the students and worsen their learning experience, so it is important to choose only a limited number of them.

Different aspects of e-learning can be examined in this way. A study [9] suggests the type of feedback delivered after each exercise can significantly impact the students' motivation. Apart from being immediate, feedback can take various forms. The study reveals the most potent form is providing not only a confirmation of the result (correct/incorrect), but also explaining why the answer is correct or incorrect, increasing both the students' motivation and objective performance in tests more significantly than other feedback approaches.

Another study [10] compares the effects of gamification and personalization in e-learning environments. By gamification, it means immersion practices such as avatars, profiles and storytelling/role-play, social features (teams, cooperation, competition) or the more achievement based techniques such as leaderboards, challenges, points and rankings.

Personalization means tailoring content to the students' individual needs by providing self-paced environment, personalized feedback from instructors or personalized content selection or even adaptive learning paths. This can be described as adaptive learning. The recommendations can be based on user data like their major or fields of interest, their behavior or in the types of mistakes they make, among other things.

The key findings of the study are that while personalization drives cognitive learning engagement, gamification promotes emotional learning engagement. Though not surprising, the implications are personalization improves learning efficiency, while gamification can be applied to keep the students engaged and persist in the process. Gamification is then more of a psychological than objective factor.

Another important finding is that different studied gamification elements have varying effect. The social elements (collaboration, competition) and immersion elements (avatars, storytelling) were of higher importance than achievement elements (experience

points, badges). It does state the achievement elements are useful nonetheless.

Though the reception of gamification is positive in almost all research papers we reviewed, some works suggest it is not so efficient in all instances[11]. However, the study aggregates many research papers across various fields, target audiences and inconsistent gamification implementations. Despite potential issues with the study, it shows that gamification is not automatically an improvement to study materials and its effectiveness depends on context and its implementation.

This point is further supported by Luo et al.[12]. It sums up other research in which gamification was not implemented successfully. The failures are linked to shift from intrinsic motivation (interest in learning) towards earning points and rank. Other potential issue was poor implementation of the gamification elements, making the students more confused than motivated.

Luo et al.[12] mentions that among the most significant barriers to gamification implementation were “Inappropriate or superficial use of gamification techniques” and “Poor integration of game features and pedagogical content”.

Humor too as a part of e-learning materials has the potential to enhance the students’ motivation and learning outcomes [13].

3.4 Applying E-learning to UML Class Diagrams

3.5 Learning Management Systems (LMS)

LMS have become an increasingly popular choice for educational institutions and individual educators seeking flexibility, transparency, and cost-effective solutions. These platforms offer a wide range of features to support online learning, from course management and content delivery to assessment and communication tools.

In this section, we briefly examine several prominent open-source LMS platforms, evaluating their general characteristics, strengths, and limitations. Each system offers a unique set of features. We will perform a comparison based on our assumed requirements that will be described later in section.

Moodle¹

Moodle is a widely-used and feature-rich open-source learning management system that supports a broad range of teaching styles and learning needs. It offers powerful tools for content delivery, assessment, tracking learner progress, and supporting collaborative learning through forums and wikis. Thanks to its extensive plugin ecosystem,

¹“Moodle,” *Moodle Project*, <https://moodle.org/>, accessed May 25, 2025.

Moodle supports gamification, microlearning, adaptive learning, and integration with external tools. While highly customizable, its interface and complex configuration may feel overwhelming to users seeking simplicity.

Open edX²

Open edX is a scalable and modular LMS originally developed by Harvard and MIT. It is especially strong in supporting self-paced and adaptive learning through structured content, video lectures, quizzes, and analytics tools. Its powerful authoring tool, Studio, allows course creators to design interactive and engaging learning experiences. While Open edX is ideal for research-driven or large-scale deployments, it has a steeper learning curve and requires more technical resources to deploy and maintain.

Chamilo³

Chamilo is an open-source LMS focused on ease of use and fast deployment, making it suitable for institutions prioritizing simplicity over advanced customization. It includes built-in tools for course management, assessments, certification, and collaboration. Chamilo enables tracking student progress and provides basic support for gamification and interactive learning. Its clean interface and low technical demands make it a solid choice for smaller teams or pilot e-learning projects, though it offers fewer integrations than larger platforms.

ILIAS⁴

ILIAS is a mature, flexible LMS that supports complex learning scenarios, including competency-based education, learning paths, and detailed tracking of user progress. Originally developed for academic institutions and public organizations, it emphasizes structure, reusability of content, and rich assessment tools. ILIAS also includes collaborative tools such as blogs, forums, and group workspaces. While powerful, its interface can appear outdated and may require configuration to meet modern usability expectations.

Canvas LMS⁵

Canvas LMS offers a modern, user-friendly experience with a focus on intuitive design and ease of use. Developed by Instructure, its open-source version includes core features such as course creation, grading, analytics, and communication tools. Canvas supports REST APIs, making it easy to extend or integrate with external tools like custom diagram editors or assessment modules. Though some advanced analytics and AI features are exclusive to the commercial version, the open-source release remains highly capable and suitable for most academic settings.

²"Open edX," *Open edX Project*, <https://openedx.org/>, accessed May 25, 2025.

³"Chamilo," *Chamilo Association*, <https://chamilo.org/en/>, accessed May 25, 2025.

⁴"ILIAS," *ILIAS Open Source e-Learning*, <https://www.ilias.de/en/>, accessed May 25, 2025.

⁵"Canvas LMS," *Instructure*, <https://www.instructure.com/canvas>, accessed May 25, 2025.

Overall comparison

Feature	Moodle	Open edX	Canvas LMS	ILIAS	Chamilo
Simplicity (UI/UX)	Moderate	Moderate	High	Low	High
Progress Monitoring	Yes	Yes	Yes	Yes	Yes
Customization	Very High	High	Moderate	Moderate	High
Ease of Content Extension	High	High	Moderate	Moderate	High
Gamification	Yes (plugins)	Limited	Yes (plugins)	Yes	Yes
Microlearning	Yes	Yes	Moderate	Yes	Yes
Adaptive Learning	Yes (plugins)	Yes	Limited	Limited	Moderate
Community and Plugin Availability	Very Large	Large	Moderate	Moderate	Growing
Feedback Channels	Yes	Yes	Yes	Yes	Yes
Ideal Use Case	Schools, Universities	MOOCs, Corporations	Schools, Universities	Corporate, Government	Schools, NGOs

Table 3.1: Comparison of selected open-source Learning Management Systems

Since the experience of studying computer science contains a large amount of self-study in any case, there are many online resources and courses in the field. Some of them utilize the techniques mentioned in the previous section. We investigated how they were used in more detail to determine whether such practices could be relevant for our intended course.

3.5.1 Gamification

Gamification is the most commonly mentioned practice when it comes to research of software modeling e-learning, and there are numerous attempts at applying it in the context of teaching the UML class diagram. The goal is to increase the students' interest and motivation in the given tasks by exploiting their natural drives.

The practice of gamification involves several techniques. Cagnazzo et al.[4] mark a student's accomplishment by levels - the student starts at level one and is awarded experience points for completing exercises. The amount of experience points gained is affected by the difficulty of the given exercise as well as the number of mistakes made by the student, providing additional motivation to reduce the mistakes for the student. Af-

ter gaining enough experience points, the student's level increases. The study proposes even the introduction of quests to further engage the learners.

The same study also implements a customizable avatar that represents the student in the application. They can alter the avatar's appearance by changing its clothes, hairstyle and other visual features. Completing exercises unlocks new styling options, again increasing the student's motivation. Lastly, the tool immediately displays feedback to the student by highlighting mistakes in various colors and by changing the student's avatar's facial expression based on the student's performance.

In another work, Jurgelaitis et al.[3] implement, in addition to previously mentioned levels and experience points, badges that are awarded for accomplishing specific goals and coins that can be exchanged for certain items. The application displays immediate feedback after completing tasks and visualizes the student's progress. Leaderboards with students ranked by their experience points are also included, invoking the students' natural competitiveness.

There are other options to incorporate the concepts of gamification into the teaching of modeling. Besides some of the already described techniques, Bucchiarone et al.[14] connect modeling with the hangman game - each mistake draws another part of the hangman, until either the exercise is completed or the hangman picture is finished (which means the player has lost).

3.5.2 Microlearning

Microlearning is the practice of breaking the study material down into small, bite-sized pieces so as to avoid overwhelming the learners. Ramle et al.[2] mention that the Learn section of their application divides the study materials into smaller sections. The general reception of the application was positive and students claimed that it helped them understand the concepts better. That suggests that applying microlearning to the topic of UML may improve the students' learning experience and possibly their resulting knowledge as well.

However, it should be noted that, even though microlearning improves motivation and knowledge retention, it is more suitable for early stages of learning when basic concepts are explained. It might not be the ideal approach when tackling more complex concepts or when constructing the bigger picture out of the smaller lessons⁶. Therefore, microlearning should not be applied to every concept, but rather to the earlier lessons only.

⁶Christopher Pappas. "Microlearning in Online Training: 5 Advantages and 3 Disadvantages." *eLearning Industry*, March 15, 2016. Available at: <https://elearningindustry.com/microlearning-in-online-training-5-advantages-and-3-disadvantages>. Accessed May 15, 2025.

3.5.3 Social learning

Ramollari et al.[15] designed a system in which students were encouraged to discuss and share their solutions to modeling exercises. Besides discussions and comments, the students could attach their solutions to other students' work as hints, alternatives or suggestions for improvement. The efforts were evaluated positively.

3.6 Common issues in Learning UML

There are several common challenges students face when learning UML. Some of them are not encountered by students exclusively but also by graduates and practitioners.

Fresh computer science graduates are expected to possess at least a basic understanding of the use of UML in software development, but that is not often the case [16]. In addition to that, even more experienced practitioners or programmers commonly claim to know UML, but later prove to only have a very limited or even wrong understanding of the concepts⁷. UML is generally considered complex and hard to grasp for novices, which contributes to this problem [17].

There are multiple research papers describing the process of identifying errors made in class diagrams. The studies [18], [19], [17] made catalogues of such mistakes, allowing easier identification of errors and potentially using them as educational materials. Chren et al.[19] revealed that the most common mistakes were related to associations, lack of domain understanding, applying inappropriate levels of detail, placing operations into wrong classes or incorrect use of inheritance.

These studies highlight the need to improve the way UML tend to be taught in most universities, as the students' knowledge often appears insufficient. Areas for improvement include providing more practical exercises, immediate feedback, increasing students' engagement, or connecting modeling with coding.

⁷Donald Bell, "An Introduction to the Unified Modeling Language," *IBM Developer*, <https://developer.ibm.com/articles/an-introduction-to-uml/>, accessed February 23, 2025.

Chapter 4

Analysis summary

4.1 Related work

There are multiple studies that tried to create e-learning modules or tools aimed at enhancing the teaching of the UML class diagram. They applied various approaches and ideas to the task.

Automated diagram assessment tools

Much of the other researchers' efforts were aimed at creating modeling tools that would enable students to draw their own class diagrams from scratch and then automatically evaluate their submissions. The goal is to provide fast and personalized feedback to the students and reduce the teachers' workload.

In these studies, the requirements for those tools were fairly similar; the drawing tools should be simple and intuitive and should be able to export the created diagrams in a format compatible with the evaluation algorithm (XMI in most cases). Then the diagram would compare the student's solution with a premade example solution and list errors or recommendations for improvement for the student.

One of the main issues with automated class diagram assessment is the fact that a modeling task is very likely going to have multiple correct solutions. One way of dealing with this issue is to store a number of possible solutions for each given problem and use the solution that is most similar to the student's current submission [8]. However, it is also possible to only use one sample solution. Doing so presents the risk of providing misleading feedback, missing errors or marking correct options as errors. In practice, these issues do not seem to be significant [20]. A less common approach to diagram quality assessment was using machine learning which achieved promising results, though only provided grading of the diagrams without specific feedback [21].

Modi et al.[22] described a tool for drawing and automatically assessing submitted diagrams. Unlike most other tools of the kind, which only support one type of diagram, the proposed tool supports a number of various UML diagrams, including the class diagram. The study also includes a brief comparison of similar tools and concludes that such tools can decrease the time and effort needed from teachers when evaluating students' diagrams.

Schramm et al.[20] also introduced an automated tool for correcting students' diagrams by comparing them to an example solution. The results of the study did not suggest that using an automated system for correcting diagrams yields better results than relying on feedback from a teacher. However, they demonstrate that such a system can save the teachers' time and resources without compromising the students' learning process.

E-learning applied to the UML Class Diagram

There were other studies aimed at teaching the Unified Modeling Language class diagram using other techniques than an automated tool for correction of the diagrams. They created platforms with various forms of quizzes and exercises and some of them applied e-learning practices such as gamification or microlearning.

In a study performed by Ramle et al.[2], a simple e-learning application for teaching several types of diagrams was designed and tested on a small sample of students. The section dedicated to explaining the concepts contains small-sized bits of information as to avoid overwhelming the students (an example of the use of microlearning). The application was generally well received by the students and, according to a survey filled by the participants, also helped them understand UML better.

Cagnazzo et al.[4] proposes using gamification mechanics to enhance the modeling classes. It introduces a web-based tool that allows the students to draw diagrams and have them automatically evaluated for mistakes. The gamified practices include levels, where a student's accomplishment is determined by a numeric value. The level can be increased by solving exercises. Another mechanics implemented in the tool are offering the students the option to customize their avatars (a picture that represents them in the application) and visually enhanced feedback that highlights their mistakes.

There are more proposed enhancement for the teaching of UML. Almadi et al.[23] propose creating a recommender system to suggest the most relevant modeling exercises to students. Lethbridge et al.[24] describe how using tools that convert diagrams to code and vice versa in classrooms helps students understand the abstract concepts of modeling. Shmallo et al.[18] arranged an activity during which students were to find mistakes in each other's diagrams. Such an activity significantly improved both their modeling skills and motivation.

Despite often being mentioned as prominent gamification elements, the concepts of storytelling and inducing competition among participants in context of software modeling e-learning materials are not discussed in the reviewed literature.

Study	Notable Features	Key Findings
Ramle et al. [2]	A course on UML, microlearning, self-directed learning	Well structured, interactive exercises and self-paced approach in the course were well received by students and improved their understanding of UML
Jurgelaitis et al. [3]	A gamified online course used in a university software modeling class	Gamification increased both students' motivation and their average scores
Soler et al. [8]	An automated diagram assessment tool used in a classroom	The group who used the tool scored higher in an exam than the group who used any other resource of choice
Cagnazzo et al. [4]	An automated diagram assessment tool, learning platform with gamified elements	Instant feedback on modeling exercises and gamified elements improved both student engagement and modeling skills
Bucchiarone et al. [14]	Online learning tool applying gamified mechanics	Using a gamified learning tool improved motivation and results of the students. They also found using the tool enjoyable
Ramollari et al. [15]	Collaborative learning environment – students discussed, evaluated and helped each other in modeling exercises	The extensive collaboration helped the students comprehend the taught materials, found it to be an effective learning method and the feedback on the system was largely positive
Shmallo et al. [18]	Identification of mistakes in diagrams as a collaborative exercise (not an e-learning tool, however)	Searching for mistakes in each other's diagrams and discussing them both engaged the students and improved their modeling skills
Schramm et al. [20]	An automated diagram assessment tool	The proposed automated diagram assessment tool saved teachers' time and energy without reducing the quality of provided education
Almadi et al. [23]	Recommender system to propose exercises to students	Personalized exercise recommendations help students better understand UML and promote self-directed learning
Lethbridge et al. [24]	Modeling tool that converts diagrams to code and vice versa	Students understand UML better when they see code corresponding to the diagrams

Table 4.1: Summary of notable e-learning tools and methods for teaching UML and software modeling

4.2 Summary

UML is a language used for modeling systems from various perspectives. The class diagram is the most prominent of the UML diagrams. It is used in various stages of system development as well as for documentation purposes.

For these reasons UML is a common part of the curriculum at universities related to computer science. Despite its inclusion, it is often observed that students as well as graduates and even professionals struggle with UML. To solve this problem, it is a worthwhile effort to improve the education of software modeling in universities.

Attempts have been made to improve the situation with the use of e-learning - the use of digital technologies to deliver educational content. E-learning can provide numerous benefits such as increased student engagement, instant personalized feedback or saving the time and attention of teachers. It is also possible to use e-learning as an addition to traditional classrooms rather than their replacement - such practice is called blended learning.

There are numerous commonly used e-learning practices with various intended benefits. The ones most suitable for educational content related to software modeling are gamification, microlearning, blended learning and social learning.

We analyzed several studies that implemented these techniques in UML learning tools. The most popular technique was gamification, where the tools support functions resembling games. Examples are giving experience points or some sort of currency for completing exercises and allowing students to create customizable avatars to represent them in the learning environment.

Most of the studies were aimed at creating tools to enhance the in-person classes at universities, not to replace them. Some of them presented courses with study materials, practical examples and tests; some introduced new collaborative learning activities, and some studies proposed tools for automated diagram assessment.

The results of these studies were predominantly positive, showing improvements both in students' scores and their engagement with the course topics. Another benefit of the described tools is that they free up the educators' time and energy that they would otherwise spend reexplaining concepts and correcting exercises.

These findings clearly document the potential of introducing e-learning tools to enhance the teaching of UML in university courses related to software modeling.

While e-learning techniques showed mostly positive results on the teaching of software modeling in the reviewed studies, they were not compared to each other within the same settings. The studies suggest techniques such as gamification or microlearning have a

potential to improve learning outcomes, but those techniques can be implemented in many forms. This presents a gap in the field of e-learning aimed at software modeling.

Based on the analysis, we decided to create an online course aimed at the UML class diagram. It will implement some of the e-learning techniques, most notably gamification and microlearning, and will test it out on students of a software modeling subject at FIIT STU. We will focus on the storytelling and role-playing aspects of gamification, using them to create 2 different designs of the course: a competitive one, and a playful one. Objective test results, behavior indicators, and students' subjective opinions will be evaluated and used to compare the two designs.

Depending on the reception of the project by students, further research can be performed to extend the scope of the course. Tools for enhancing the teaching of software modeling can be introduced, for example an automated diagram assessment tool. Other types of diagrams or software modeling materials can be added to the course. The use of the e-learning practices can be further extended as well.

Chapter 5

Objectives and methodology

5.1 Objectives

The goal of this study is to create an online course for teaching the UML class diagram and analyze various approaches to doing so. The target audience are computer science students taking a software modeling course at FIIT STU.

It is a project that aims to compare 2 of the most prominent approaches to enhancing study materials in e-learning - competition and play - and applying them to teaching software modeling. By comparing different course designs within the same setting, we seek to fill a gap we observed while reviewing related literature.

Additionally, we want this study to be a starting point for further educational content at FIIT STU should this project prove successful. In such a case, other researchers could extend the course by adding additional features and content, including other types of diagrams, or by adjusting the materials to try out different e-learning approaches to delivering content and driving engagement. On top of that, e-learning materials in other areas might be developed using findings in this project.

We present the following research questions based on our initial analysis and objectives:

1. Does competitive course design lead to higher learning performance compared to playful design?
2. How do playful and competitive course designs influence students' behavioral engagement?
3. How do students evaluate playful and competitive course designs in terms of emotional engagement, satisfaction, and motivation?
4. What are students' perceptions of the usefulness and future potential of e-learning

materials at FIIT STU?

5.2 Methodology

The courses will be implemented in an established e-learning platform. They will contain theoretical materials, interactive exercises, self-assessment components, as well as e-learning practices based on our analysis. Both courses will contain the same exercises and materials, but they will differ in tone and use of e-learning practices. That will allow for separate analysis of the educational approaches.

The courses will be taken by second-year students of computer science who will be presented with 2 tests and 2 questionnaires - one of each at the start, and one of each at the end of the courses. They will be used to measure the students' interest and level of knowledge before and after taking the course. Their behavioral patterns while taking the courses will be measured and evaluated as well. Each student will only take 1 of the courses.

The tests will be made up of theoretical questions and practical exercises, while the questionnaires will be aimed at the students' subjective reception and interest. The most important questions will target the students' interest in such materials and whether they think it is an improvement over the standard way of teaching software modeling.

The students will be divided into two groups: one for each of the courses. Both groups will be given the same tests and questionnaires to determine their skill level and subjective opinions. We will gather results, analyze them and draw conclusions based on them.

Chapter 6

Design

6.1 Solution description

When designing our intended class diagram courses, we will take into account a number of requirements. Firstly, we have the platform or LMS on which to implement the course, then there are e-learning techniques to use, specific types of exercises and other educational content we want to apply, and lastly we need to propose an effective and objective evaluation method to determine the potential of the project for further research.

Additional constraint that we need to account for is the scope of the course. Even though an ideal UML course might guide the user through building an entire system, this course is complementary and the software modeling class's curriculum does not make space for it. Participation in this course is voluntary. For these reasons, we decided to restrict the course's scope to only one diagram type to test out the concept, and to fit all the content into one hour.

The UML class diagram was preferred over other diagram types, because it is one of the most prominent UML diagram types - likely the most widely used one - and it is intuitive to understand. It directly links to coding which students tend to comprehend better than abstract diagrams [24].

6.1.1 E-learning platform

We want to create the courses on a preexisting LMS. Doing so will significantly reduce development time, allowing us to focus on the educational content.

The platform should satisfy the following conditions:

- easy to use and learn

- record and store students' scores and progress for result comparison and material recommendations
- option to propose changes and improvements, report mistakes and bugs
- easy to include new features and content
- free to use or open source
- support for e-learning techniques described in the next sections
- support for interactive exercises

6.1.2 Content Structure

This section is aimed at describing what specific types of content the course should contain. We will divide it into four sections - an introductory test to assess the starting point of the students, a theoretical section dedicated to explaining concepts, one for practical exercises, and one for final skill assessment after taking the course. The final test will be accompanied by a questionnaire.

1. Introductory assessment test
2. Theory
3. Practical exercises
4. Final assessment test, questionnaire

Activity diagram - student

The activity diagram describes a student's interaction with the system. The student will log in to the system and view a course. They will complete the introductory test. Then they will proceed to the theoretical section with micro-exercises in between modules and the practical section containing more exercises. Content will be recommended based on how successful their responses were across various topics. The student can come back to the theoretical section and the exercises and redo them until they are satisfied with their results. Lastly, the student will enter the evaluation section, where they will complete a test (which will not give them instant feedback this time), and fill in a feedback questionnaire.

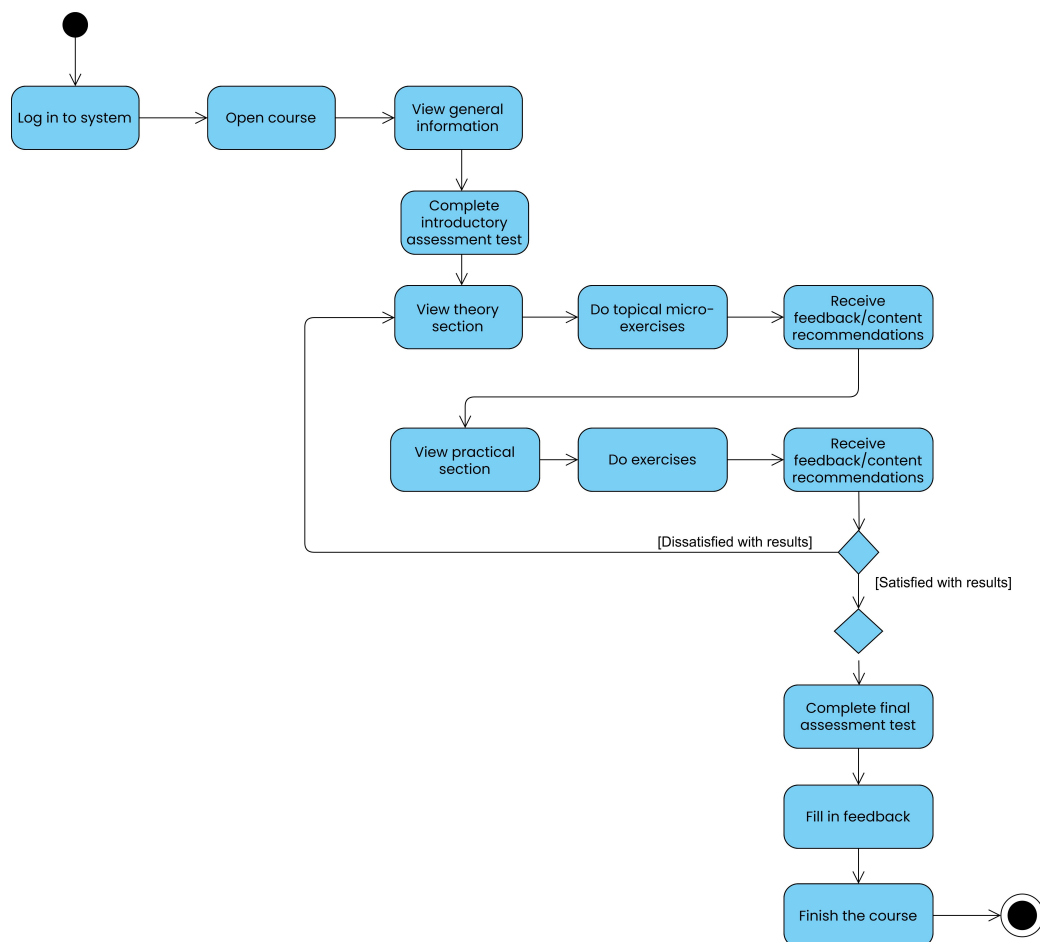


Figure 6.1: Activity diagram from a student's perspective.

Made with: Visual Paradigm, *Visual Paradigm Online*, <https://online.visual-paradigm.com/>, accessed Dec 9, 2025.

6.1.3 Content - Theory

The theoretical section will contain a set of chapters each dedicated to a single concept or component of the class diagram. The concepts will be added gradually, simulating the process of building a diagram from scratch, while describing modeling decisions at each step. There will be micro-exercises placed between theoretical sections.

Since students understand UML better when they see corresponding code [24], there will also be a snippet of code available for diagram demonstrations.

6.1.4 Content - Exercises

A limitation of the exercise design is the absence of a UML diagram editor. We do not have the capacity to employ teachers to manually assess modeling exercises. At the same time, designing a comprehensive automated diagram assessment tool is a non-trivial problem and is beyond the scope of this project.

For these reasons, we will replace direct modeling tasks with a set of exercises implemented using more general, readily available formats. The exercise design is inspired by findings from our analysis, namely:

- Searching for mistakes in others' diagrams can enhance the students' modeling skills [18]
- Students develop a better understanding of UML when they see diagrams translated to code [24]
- Exercises work best when feedback is immediate and explains why the answer is correct or incorrect [9]

They will be of the following types:

- Fill in missing parts of a diagram (relationships, multiplicity...). This is the most important exercise type, since it directly simulates modeling
- Assign the type of relationship to a description or a code snippet
- Find mistakes or improvements in a diagram
- Decide, which of the diagrams contains a mistake
- Compare a diagram with a code snippet
- Alter a diagram based on instructions
- Match class diagram components with their descriptions and pictures
- Theoretical questions

The sections with tests will consist of a number of exercises selected from across all the course's topics. The results will be recorded and analyzed later.

6.1.5 E-learning techniques

Based on our analysis, we concluded that the most suitable e-learning techniques are microlearning, gamification, adaptive learning and social learning. Our courses will also apply self-directed learning, since a course can be taken at any time and at any place, and blended learning, as it is meant to be a complementary tool to normal classes rather than a stand-alone learning material.

Under microlearning we understand breaking down content in the sections dedicated to explaining the concepts into small pieces.

Gamification offers the largest amount of possible features in an online course out of all the e-learning techniques.

It is important to note that there is a limited number of gamified elements that can be implemented. One of the main constraints on them is that they must not overwhelm or confuse the students. The one-hour format of the courses does not provide the space to use more complicated mechanics associated with gamification, as they might take time to get used to. It was observed that some gamified courses slowed the learners down first before increasing their performance due to the need to adapt to the different format of the study method [25].

Examples of such techniques are levels, experience points, purchasable items within the course or meaningful quests, which are common instances of gamification. Instead, we want to use storytelling and role-play, which might be even more efficient than artificial awards such as points [10]. We will use the role-play as the main distinction between the 2 versions of the courses, giving the narrator of the instructions and theoretical sections a different personality - one will be fun and warm, the other brief, demanding and professional. We want to further enhance the mood of the courses by selecting appropriate - but applicable - e-learning techniques.

The 2 moods for the course were chosen because competition and fun are among the most commonly used gamification elements. Fun is associated with rewards, humor, emotional ease, and playfulness, while competition aims at self-accomplishment and seeking achievements. That makes the fun, playful design and the competitive design ideal for comparison.

The competitive version of the course will contain a leaderboard and display metrics of other students (more on that later in the same section). We decided not to use a timer for tasks even though it would fit the mood of the course, since it presents hindrance for students with lower base knowledge and encourages rushing through materials.

The playful version of the course will strive to increase the students' emotional engagement by providing visual rewards for correct answers. Another element to complement

the design are badges awarded for reaching milestones within the course, which are a popular addition to gamified environments - despite not being sufficient to engage students on their own. Lastly, students will be given funny nicknames to attract their attention at the start of the course.

On top of that, we plan to implement visual changes triggered by progress in the courses, mistakes or difficulty level of exercises.

Adaptive learning should allow the course to propose content to the student based on their mistakes in exercises. Feedback should be specific and explain why an answer is incorrect.

When it comes to social learning, collaborations on modeling tasks are not allowed by the experiment format - students will take the courses on their own outside of classes. For this reason, we will restrict the practice to displaying a number of metrics to the users, such as the average score on a given exercise, average completion time or the overall progress of other students in the course. Doing so will apply some social pressure on the students to perform better and potentially trigger their competitive tendencies. This practice is sometimes mentioned as a part of gamification.

Since such practices are sometimes questioned in literature, we will restrict this functionality to the competitive version of the course.

E-learning techniques that are to be implemented:

- microlearning
- gamification (narration, badges...)
- adaptive learning (feedback, content recommendations)
- social learning (performance comparisons, leaderboards - restricted to competitive course)
- self-paced learning
- blended learning

6.1.6 Evaluation

Tested students will be divided into 2 groups - each student will only take one of the course versions. That will allow for separate analysis of effects of the different course designs on the students.

The first part of evaluation will consist of a questionnaire with questions about subjective opinions and impressions of the students. The most important questions will concern whether the students thought the course was helpful, whether it improved their understanding of the class diagram and, most importantly, how the e-learning elements impacted their perceptions and behaviors during and after taking the course.

The second, objective part of the evaluation will consist of 2 tests containing theoretical questions and practical exercises about the class diagram. One of the tests will be taken before completing the course, the other one after. They will be used to evaluate the progress of the students. The same tests will be taken by both groups.

The last part of the evaluation will measure the students' behavior - such as time taken to complete activities and the amount of interactions with the system's elements under each course design.

6.2 Requirements

From the solution description, we can extract lists of functional and nonfunctional requirements.

Functional requirements

- Automated exercise/test correction with and instant feedback
- Support for:
 - Gamification- visual feedback, difficulty indicators, badges...
 - Microlearning
 - Adaptive learning - content recommendations based on a student's results, feedback
 - Social learning - displaying other students' performance (anonymized)
- Support for various interactive exercises and learning materials
 - UML code snippets linked to diagrams
 - Full diagram examples with explanation of modeling decisions
 - Drag and drop exercises
 - Matching exercises, multiple choice exercises
 - Theoretical questions
- Students need to provide anonymous feedback on the content
- Tests of students' skills
- Monitoring of students' behavior

Nonfunctional requirements

- Easy to learn and use
- Open source or free to use
- Capable of storing and retrieving student data
- Scalable and maintainable for future expansion
- Easily extensible (adding new features/content)
- Restricted access - allow access only to students and teachers at FIIT STU
- Content can be accessed anytime, anywhere

Class diagram of the system

This diagram describes the entire system as we intend it to function. It shows all the major entities that should be found there - users, course and its content, tools for tracking progress and modules for additional features such as adaptive learning.

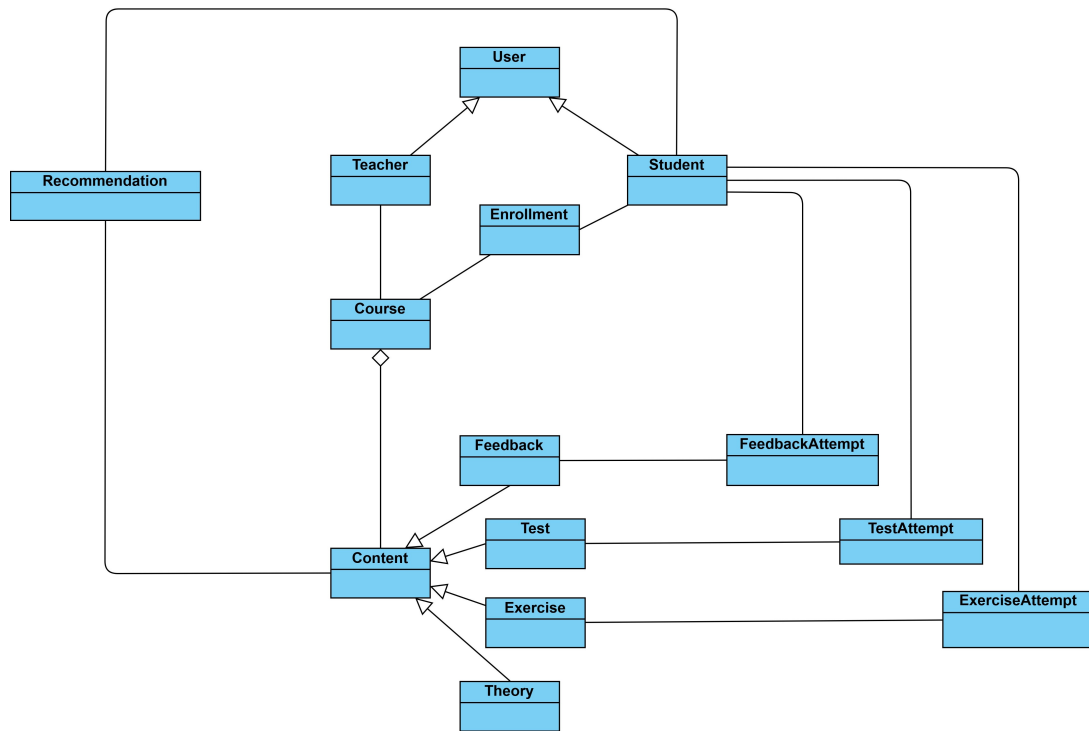


Figure 6.2: Class diagram of the designed system.

Made with: Visual Paradigm, *Visual Paradigm Online*, <https://online.visual-paradigm.com/>, accessed Dec 9, 2025.

Sequence diagram - entire system

The sequence diagram describes the interaction between all the major entities in the system - teacher, student, user interface, backend. Steps such as authentication were omitted for better clarity. The teacher needs to create courses and their contents. Then the student can enroll in a course and complete it in the flow already described in the activity diagram (see figure 6.1). Lastly, the teacher can review the student's results and make their conclusions and adjustments.

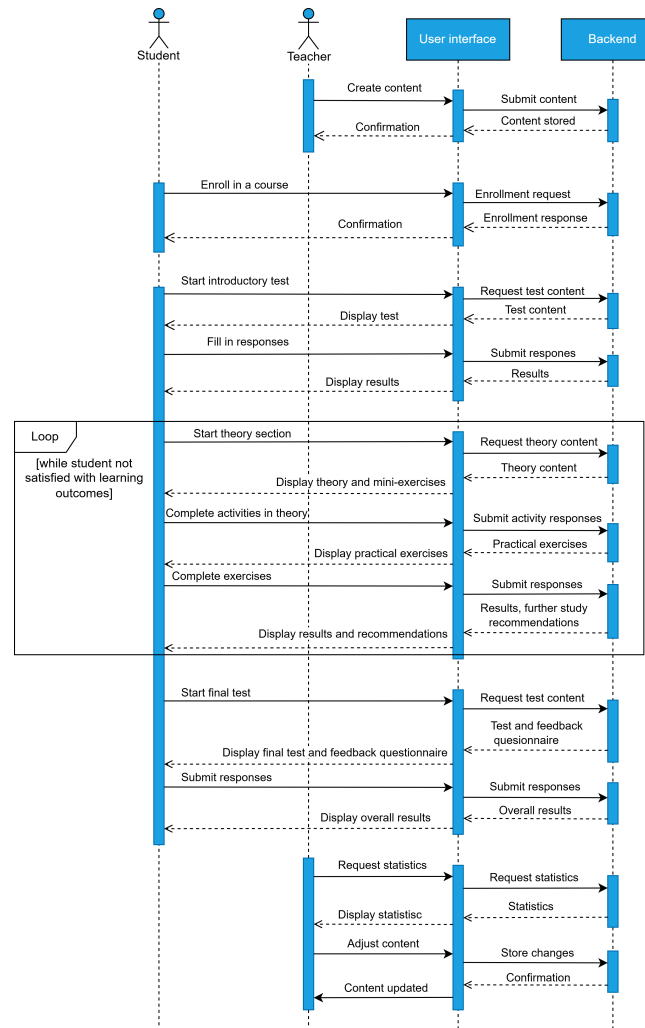


Figure 6.3: Sequence diagram of the whole system's workflow.

Made with: Diagrams.nte, *Diagrams.net*, <https://app.diagrams.net/>, accessed April 30, 2026.

Chapter 7

Implementation

We decided to implement our course in Moodle LMS. Moodle satisfies the vast majority of our requirements - both functional and nonfunctional.

It is open source, easy to use with an extensive documentation and community support, and it is possible to deploy on local servers, allowing us to control the amount of computational resources dedicated to it for future scaling. We can restrict access to a set of users defined by us. It stores information about students' progress, generating statistical reports about their behaviors and results automatically. Content can be added easily, with a lot of plugins developed and maintained by community that introduce features not present in basic Moodle itself.

Moodle supports most of the proposed exercise types out of the box - matching exercises, code snippets, drag and drop, multiple choice, and other. Exercises are corrected automatically, feedback can be provided for each answer in an exercise. Tests and feedback forms are also implemented and ready to use in Moodle.

The biggest hindrance is the absence of content types designed for UML education. However, we found no other LMS that would provide this functionality without the need for a workaround, so that is not a decisive issue. For the first version, we decided to substitute the need for an editor with drag and drop exercises.

7.1 Course content

More examples from the course are attached in Appendix A.

7.1.1 Theoretical content

We placed the chapter with explanations of concepts of the class diagram right after the introductory assessment quiz and questionnaire. It is implemented as a number of Moodle's book resource¹ instances.

The book resource contains a number of pages with static content. It is structured like an actual book with a table of contents, chapters and subchapters. This way the student is aware and in control of the flow of their progress. It can be exported and printed to a .pdf file for future reference by the students as well.

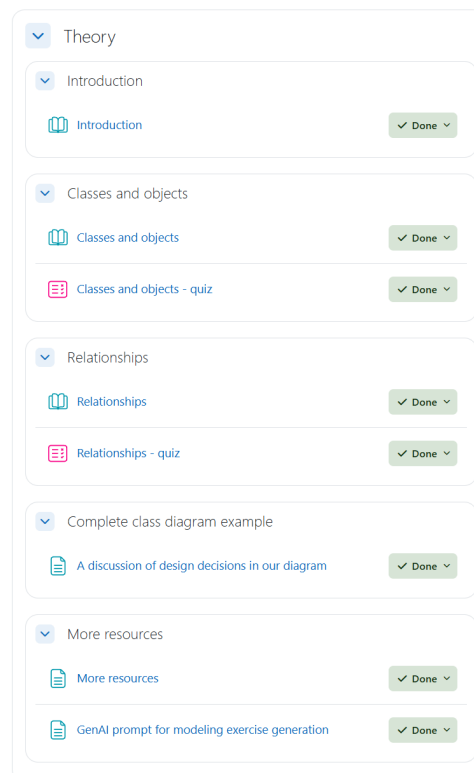


Figure 7.1: Navigation of the theory section

It begins with an introduction that explains the importance of software modeling and what UML is.

The following sections guide the students through the process of building a class diagram -a simple model of an e-shop- one component at a time, explaining each newly added concept and discussing design choices.

¹Moodle, "Book resource", https://docs.moodle.org/501/en/Book_resource, accessed December 13, 2025.

The first book contains chapters about these concepts: entities, classes, objects, types, methods, attributes, access modifiers. It is followed by a short quiz with exercises related to the topics.

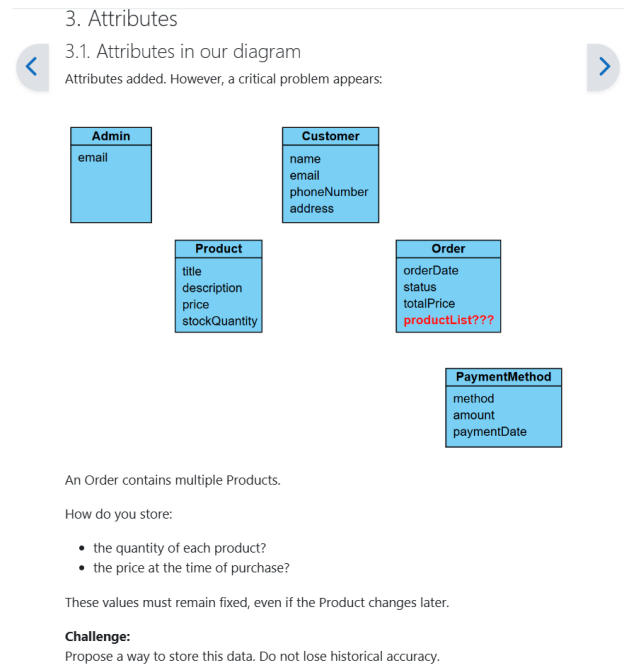


Figure 7.2: Gradually building a diagram while discussing modeling challenges as they arise

The second book talks about relationships - various types of relationships, differences between them, how they can be added to the example diagram, multiplicity and role names. This book has an associated quiz with a few related exercises as well.

The section continues with a picture of the example diagram and a list of modeling decisions the students might question while reading the theory. Those questions include: “Should Admin be a separate class?” or “Why are there no ids, no Product in OrderItem, no getters or setters?”

5. Why are there no ids, no Product in OrderItem, no getters or setters...

Implementation uses them—every class will have an ID.

- **Omission rationale:** UML focuses on conceptual clarity, not code-level detail.

Conclusion: Include only what adds value for understanding [relationships](#) and responsibilities.

Figure 7.3: Discussion of design details in the example diagram

The last part of the theoretical section is a short collection of links to more resources about software modeling - a full course, video, article, a link to more examples. There is also a prompt for Generative artificial intelligence (Gen AI) in case students want to easily create exercises of their own.

There were other options to implement this section than the book resource, mainly Moodle's Lesson activity², which provides more space for adaptive learning. However, it cannot be referred to in later stages of the course, which is what made us choose the book instead. Appropriate theoretical sections are linked in feedback to exercises in case a student makes a mistake.

7.1.2 Exercises

Moodle allows us to implement the majority of the exercise types described in earlier chapters (see Section 6.1.4). We made 29 exercises in total, most of them focused on simulating modeling diagrams by using Moodle's drag and drop question type³ and exercises in which the students need to connect diagrams with code. Exercises display hints, reasoning behind correct answers, or links to corresponding theoretical sections after incorrect answers.

All exercise types are:

- Differentiating between relationship types based on description or context
- Find mistakes or room for improvement in a diagram
- Choosing the right relationship arrow directions
- Selecting entities that should be found in a diagram based on a description
- Simulating modeling by dragging components onto empty slots in an image of a diagram
- Theoretical questions - matching access modifiers to their symbols, differentiating between classes and objects...

²Moodle, "Lesson activity", https://docs.moodle.org/501/en/Lesson_activity, accessed October 7, 2025.

³Moodle, "Drag and drop onto image question type", https://docs.moodle.org/502/en/Drag_and_drop_onto_image_question_type, accessed December 19, 2025.

Chapter 7. Implementation

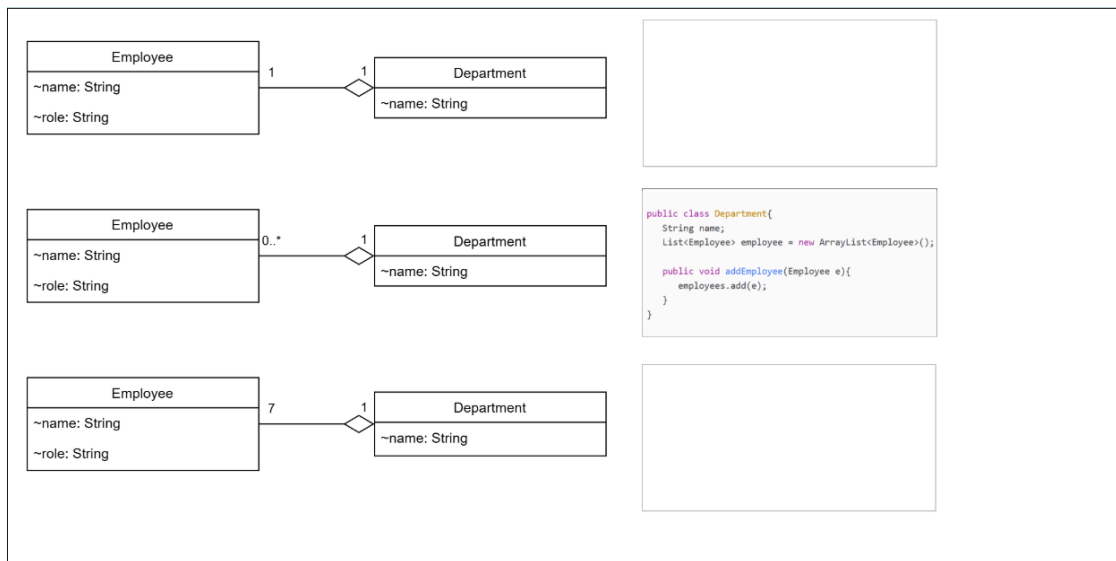


Figure 7.4: Exercise for matching code with corresponding diagrams

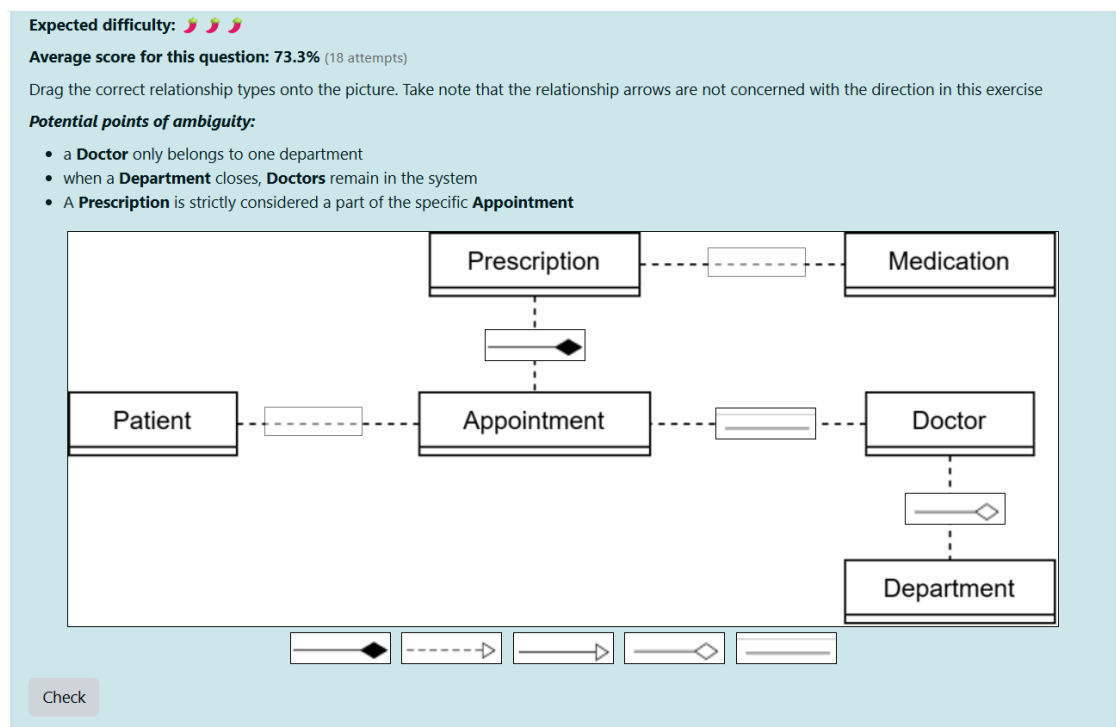
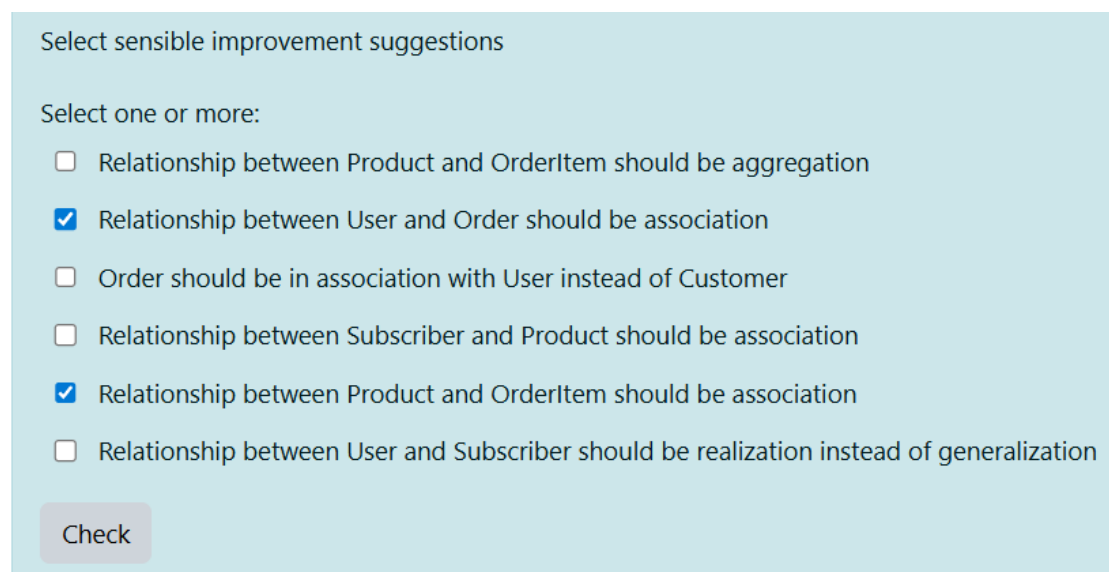


Figure 7.5: Simulating modeling with the drag and drop onto picture activity



Select sensible improvement suggestions

Select one or more:

- ☐ Relationship between Product and OrderItem should be aggregation
- ☒ Relationship between User and Order should be association
- ☐ Order should be in association with User instead of Customer
- ☐ Relationship between Subscriber and Product should be association
- ☒ Relationship between Product and OrderItem should be association
- ☐ Relationship between User and Subscriber should be realization instead of generalization

Check

Figure 7.6: Picking correct improvement suggestions for a flawed diagram

7.1.3 Testing students' knowledge

Moodle's quiz activity⁴ is a selection of exercises from a defined set. It is possible to have static quizzes that contain the same exercises every time, or select a number of exercises at random from a set.

We used the quiz activity for all the exercise sections. In the quiz settings, the practice sections show instant feedback for each response in each exercise after the student clicks a check button found under each exercise. They are allowed to retry each exercise multiple times, getting hints in between attempts as well. However, retrying a question multiple times reduces the amount of points received for the question.

In the final assessment and introductory assessment sections, the immediate feedback is not present and number of attempts is lower.

7.1.4 Statistical reports

Moodle provides statistics regarding individual quizzes, exercises and the overall success of individual students⁵. Reports contain statistics like effective weight or discrimination index mainly useful for adaptive learning practices. We created our own reports aimed at comparing our 2 courses.

⁴Moodle, "Quiz activity", https://docs.moodle.org/501/en/Quiz_activity, accessed October 7, 2025.

⁵Moodle, "Statistics", <https://docs.moodle.org/501/en/Statistics>, accessed December 13, 2025.

Chapter 7. Implementation

We used the Configurable reports plugin⁶ to perform queries on the Moodle database. The plugin allowed for convenient .csv exports of the reports.

Some of the reports were used as dynamically updated statistics for the students - they could view their performance across various categories or compare their performance with other students in the Competitive version of the course. However, most of them were used in the evaluation of the experiments.

Manage reports / Intro vs final quiz improvement

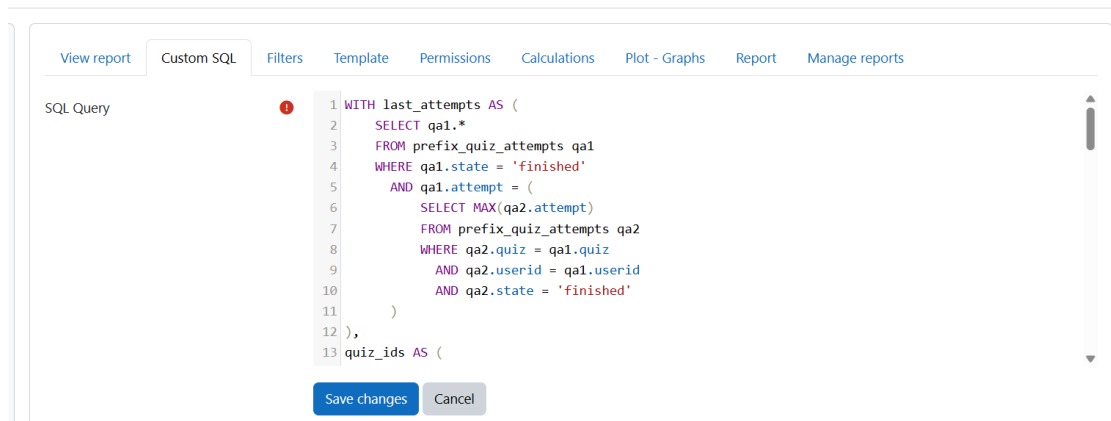


Figure 7.7: Creating a custom report.

The screenshot shows the 'Report' tab in the Moodle Configurable Reports plugin. The report displays a table with the following data:

nickname	intro_score	final_score	improvement
subject19	69.49	92.51	23.02
subject13	67.82	79.67	11.85
subject23	76.41	84.78	8.37
subject6	83.33	90.86	7.53
subject34	91.92	98.66	6.74
subject27	83.85	90.52	6.67
subject10	82.37	86.53	4.16
OVERALL	82.83	87.02	1.47
subject14	94.87	95.67	0.80
subject35	82.69	81.68	-1.01

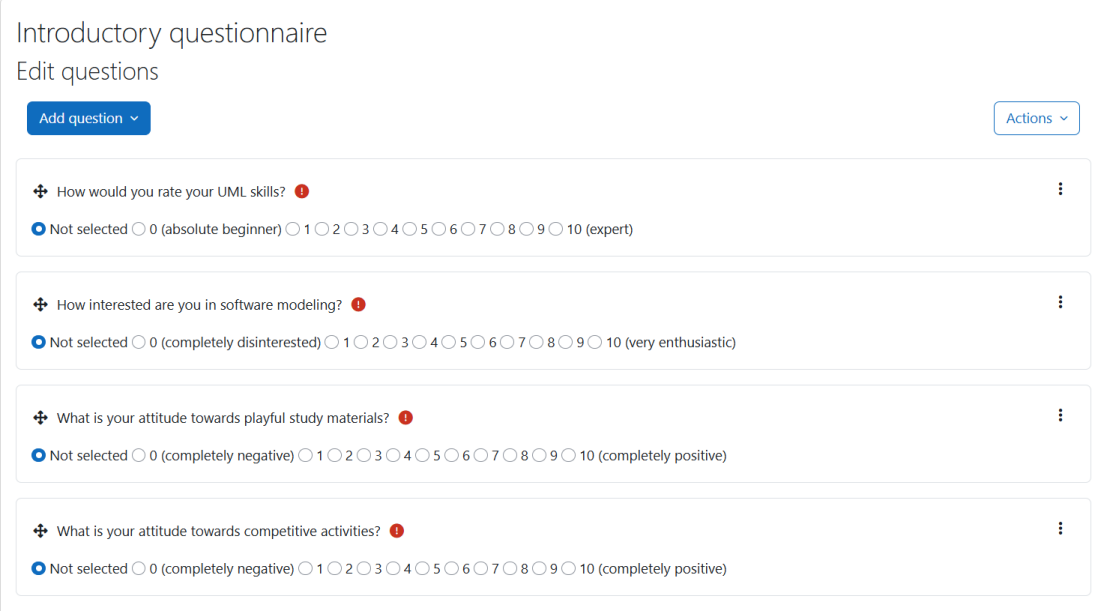
Showing 1 to 10 of 71 entries. Total record count = 71. Navigation buttons: First, Previous, 1, 2, 3, 4, 5, ..., 8, Next, Last.

Figure 7.8: Example of a custom report.

⁶Juan Leyva and Sara Arjona Téllez, "Configurable reports", https://moodle.org/plugins/block_configurable_reports, accessed February 13, 2026.

7.1.5 Feedback questionnaire

Collecting feedback from participants is one of the most important parts of the whole project. Moodle’s feedback activity⁷ lets students answer questions about their perception of the course. The responses are anonymous. Both questionnaires were made with the activity.



The screenshot shows the 'Introductory questionnaire' interface in Moodle. At the top, it says 'Introductory questionnaire' and 'Edit questions'. Below this are two buttons: 'Add question' (blue) and 'Actions' (blue with a dropdown arrow). The main area contains four question cards, each with a plus icon, a question text, a red exclamation mark icon, and a three-dot menu icon. Each card has a radio button for 'Not selected' and a series of radio buttons for a scale from 0 to 10, with descriptive text at the ends of the scale.

Introductory questionnaire
Edit questions

Add question Actions

How would you rate your UML skills? !

Not selected 0 (absolute beginner) 1 2 3 4 5 6 7 8 9 10 (expert)

How interested are you in software modeling? !

Not selected 0 (completely disinterested) 1 2 3 4 5 6 7 8 9 10 (very enthusiastic)

What is your attitude towards playful study materials? !

Not selected 0 (completely negative) 1 2 3 4 5 6 7 8 9 10 (completely positive)

What is your attitude towards competitive activities? !

Not selected 0 (completely negative) 1 2 3 4 5 6 7 8 9 10 (completely positive)

Figure 7.9: A sample from feedback questionnaire for students.

⁷Moodle, “Feedback activity”, https://docs.moodle.org/501/en/Feedback_activity, accessed December 13, 2025.

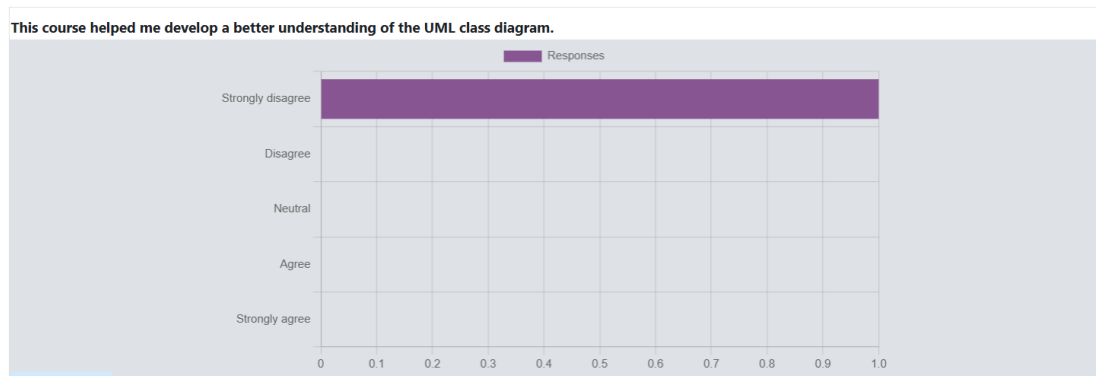


Figure 7.10: Viewing the results of the feedback questionnaire.

7.2 E-learning techniques

Microlearning - we broke the theoretical section into small chapters, where each chapter contains only 1 concept. There are also exercises placed in between theoretical sections to make the content more engaging.

Table of contents
1. Introduction
2. What is a class?
3. Attributes
4. Types
5. Methods
6. Access modifiers
7. Classes VS objects

Figure 7.11: Clear separation of topics for microlearning

Blended learning in our courses is implicit. The courses were used as a part of an actual university class, so the participants experienced both e-learning and traditional learning.

Self-paced learning - students were allowed to take the course whenever it suited them, and did not have a time limit on finishing it after they started.

Adaptive learning and instant feedback - students have access to a report that displays their performance across various categories.

tag	points_earned	points_max	score_percentage	recommendation
multiplicity	10.74	12.00	89.51	Very good – minor gaps
entity_identification	8.00	10.00	80.00	Very good – minor gaps
methods_and_attributes	18.75	24.00	78.13	Very good – minor gaps
code_vs_diagram	9.67	13.00	74.36	OK – recommend additional practice exercises
relationships	26.69	37.00	72.14	OK – recommend additional practice exercises
access_modifiers	8.33	12.00	69.44	OK – recommend additional practice exercises

Figure 7.12: A student's performance across various categories.

After answering exercises, students were shown whether the answer was correct, partially correct, or incorrect. In case of incorrect answers, they might be shown hints, links to related theory sections or rationalizations behind the answers.

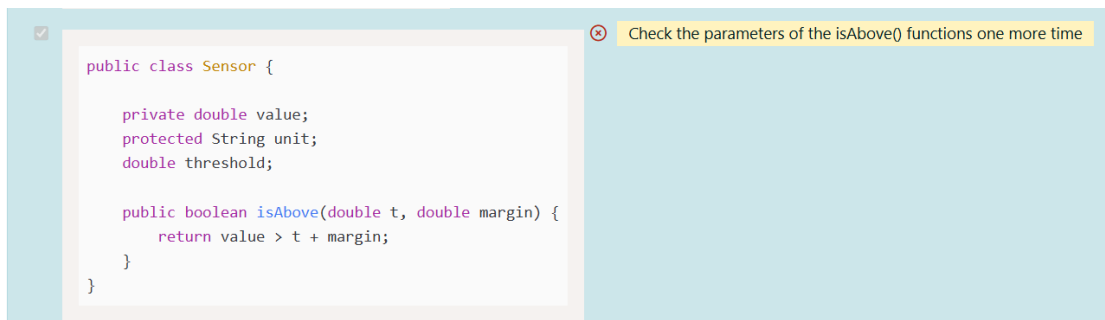


Figure 7.13: An incorrect answer displays a hint to the student.

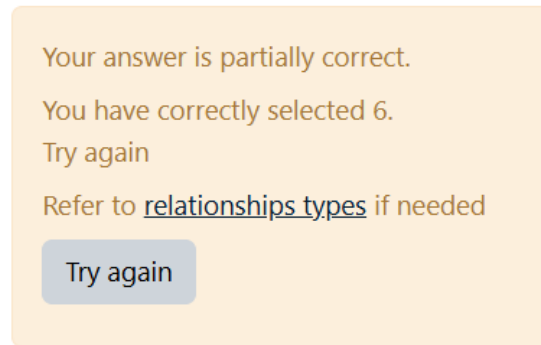


Figure 7.14: Related theory section is suggested after an incorrect answer.

Visual indicators

Difficulty of exercises was marked by visual indicators in form of chillies - more chillies indicate higher difficulty. As opposed to the design, we decided not to include visual changes to the website's interface based on the current student's performance due to complicated implementation of such an idea.

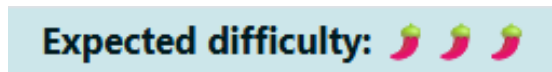


Figure 7.15: An exercise's difficulty is marked by a visual indicator

7.2.1 Playful course version

For the playful course version, we implemented several gamification practices:

- Granting rewards for correct answers
- Awarding badges for reaching milestones in the course
- An informal, playful narration of theoretical and informational sections of the course

Native Moodle functions were used to design the rewards and badges. The rewards were inserted into feedback for correct responses in exercises. The badges are awarded for reaching certain milestones in the course and a notification was sent to the user automatically once the award criteria were met.

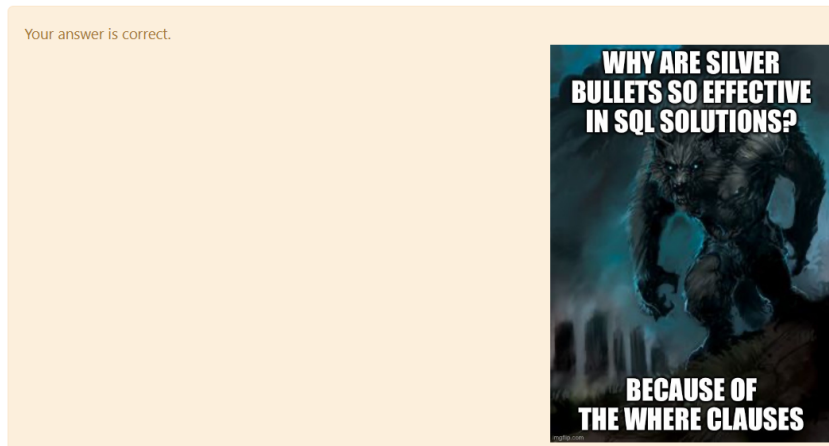


Figure 7.16: Solving an exercise correctly grants a reward

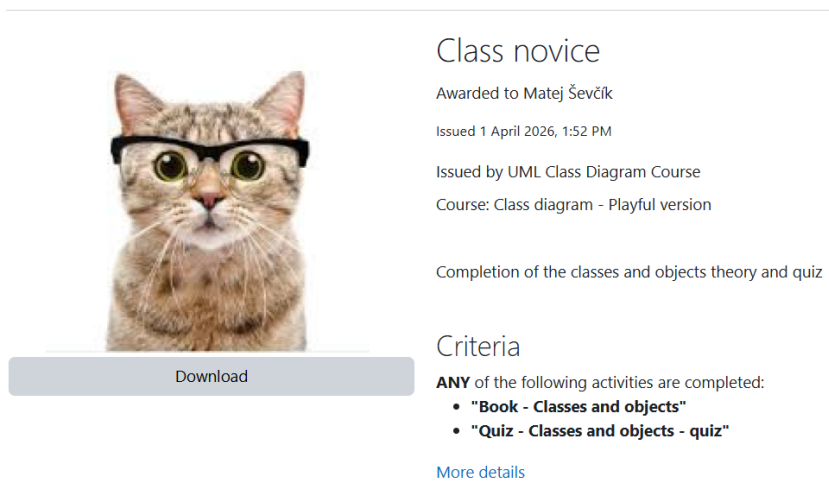


Figure 7.17: Badges were awarded for reaching certain milestones in the course

Chapter 7. Implementation

1. The importance of software modeling

Let's be honest: jumping straight into coding without a plan is **bold**. Gently put.

Software modeling helps you think things through *before* writing code. It gives you a clearer picture of what you're building and how it should work.

Why it matters:

- **Saves time & money** - Fixing a mistake in a diagram is much easier than fixing it in code.
- **Communication** - So your teammates don't interpret that "simple feature" as 5 different things.
- **Documentation** - Future you (and your colleagues) will thank you.
- **Career bonus** - Many claim to know UML, but few actually use it well. That can give you an edge...
- **Big-picture thinking** - Forces you to think before coding. Revolutionary concept, I know.

Challenge 🧐

Have you ever coded something and later thought "*this could've been designed better*"? (OOP flashbacks? No? Just me?)

Figure 7.18: Theory being explained in an informal, playful tone

7.2.2 Competitive course version

In the competitive version of the course, the following elements were introduced:

- Leaderboard
- Report that displays the student's performance across categories compared to the course's average
- Each question displays the average score of all the attempts at it across the entire course
- Assertive, strict narration of theoretical and informational sections of the course

The leaderboard displays the performance of the 10 best students across both courses. A student would also see their own standing. We used the already mentioned Configurable Reports plugin to create it.

The same plugin was used to create the comparison report which displays the exercise category, the current user's score in it, the average score in the course, and how the current student compares to the average values.

Chapter 7. Implementation

Show entries

Search:

id	nickname	finalgrade	position	percentile	bonus_eligible
51	voldemort1	97.91663	1	100.00	1
19	gladiator	97.71429	2	97.56	1
31	subject34	97.62422	3	95.12	1
89	subject14	95.99951	4	92.68	1
88	terminator	94.45834	5	90.24	1
26	subject25	93.32894	6	87.80	1
41	shadowhunter	92.79141	7	85.37	1
80	subject40	92.22168	8	82.93	1
61	warbringer	91.16460	9	80.49	1
85	thor	91.05957	10	78.05	1

Showing 1 to 10 of 10 entries

Total record count = 10

First Previous 1 Next Last

Figure 7.19: 10 best performing students are displayed in the leaderboard along with their scores

Show entries

Search:

tag	student_score	course_average	difference
OVERALL	85.11	76.06	1.12
methods_and_attributes	90.83	58.25	1.56
access_modifiers	83.33	60.90	1.37
code_vs_diagram	87.50	69.14	1.27
Difficulty: 🍌 🍌 🍌	79.41	68.35	1.16
entity_identification	90.48	78.99	1.15
multiplicity	90.77	82.60	1.10
Difficulty: 🍌 🍌	93.33	85.76	1.09
Difficulty: 🍌	98.10	95.68	1.03
relationships	77.29	80.82	0.96

Showing 1 to 10 of 10 entries

Total record count = 10

First Previous 1 Next Last

Figure 7.20: Students can compare their performance across various categories with the course's average

To display the course's average on a given exercise, we used the Generico Filter plugin⁸. It allows us to run a database query and place its formatted output inside any text on the Moodle site. Since the plugin retrieves data from the database before parsing the arguments it was passed in Moodle content, we opted for a workaround. The query calculates the average score across all exercises along with the number of attempts, and then uses Javascript to retrieve the result for the specific identifier passed as an argument to the filter.

The solution might become problematic in case of very large sites, where retrieving data for all exercises would cause performance issues.

⁸Justin Hunt, "Generico Filter", https://moodle.org/plugins/filter_generico

Average score for this question: 73.3% (18 attempts)

Figure 7.21: The course's average score is displayed when solving an exercise

1. The importance of software modeling

Software modeling defines what the system must achieve before implementation. Skipping this step leads to misaligned requirements, costly rework, and unstable designs.

In professional environments, modeling is *not optional*.

Why it matters:

- **Communication** - Diagrams replace ambiguity with precise structure. Teams that model communicate faster and with fewer errors.
- **Requirements definition** - A correct model proves you understand the problem before writing code.
- **Documentation** - Well-designed diagrams serve as long-term technical documentation.
- **Cost efficiency**
 - Errors are identified before implementation
 - Rewrites are minimized
- **Competitive advantage** - Most developers claim UML knowledge. Few can apply it correctly. This skill differentiates candidates in design-oriented roles.

Challenge:

If you cannot describe your system with a diagram, you do not fully understand it.

Figure 7.22: Theory being explained in an assertive, demanding tone

7.3 Deviations from design

Several changes were made when compared to the intended design (See Chapter 6). Some of the proposed content proved either impractical, hard to implement, or there was not enough room in the courses for the ideas to be included. The changes were:

- Course structure change - an additional questionnaire was added at the start of the courses. Originally, it was proposed to only include a questionnaire at the end of the courses
- Providing code snippets along with diagram examples in the theory section proved impractical - the diagram example corresponded to a rather large codebase, and including even parts of it would have been either insufficient or overwhelming. We only applied comparison of code and diagrams in exercises
- Visual indicators based on the student's current grade. We decided to only include exercise difficulty indicators
- Exercises type *Alter a diagram based on instructions* was excluded, as there was no way to implement without creating custom activity types in Moodle

7.4 Deployment

We were given access to a server owned by FIIT STU for the duration of the experiment (15.3.2026-4.5.2026). The specifics of the server were as follows:

- Operating system: Ubuntu 22.04 LTS ⁹
- Number of virtual CPU cores: 2
- Memory size (GB): 4
- Data disk size (GB): 50

The amount of computational resources was requested based on the assumption that no more than 80 users would use the site, and it should handle up to 20 users at a single time. The disk size may seem large for the use case, but Moodle stores a lot of user data.

For the duration of the experiment, domain `fiiture.sk` was kindly provided by Ing. Juraj Vincúr, PhD. which allowed us to set up a secure connection using Let's Encrypt¹⁰.

Instead of restricting access to the website to only allow users with a FIIT STU Virtual private network (VPN), we disallowed the creation of new accounts. That means we generated accounts to our target students who were then sent credentials via email. We did not implement any additional access restrictions.

⁹Canonical Ltd., "Ubuntu Server 22.04 LTS (Jammy Jellyfish) Cloud Image", <https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img>, accessed March 15, 2026.

¹⁰Let's Encrypt, <https://letsencrypt.org/>, accessed March 15, 2026.

Chapter 8

Experiment and Evaluation

8.1 Experiment

We tested the created courses on a sample of computer science students at FIIT STU. The students were all in their second year and attending a software modeling class. Each student was given an account on our Moodle site and was enrolled in one of the courses. We decided to assign the course randomly.

Participation in the experiment was voluntary. Students who filled in their assigned course were given bonus points (2) to their software modeling class, and those who scored in top 30% across both courses were given double the credits (4). In total, 35 students completed their course - 14 for the competitive variant, 21 for the playful variant. Some of the participants dropped out, resulting in varying amounts of activity completions throughout the courses.

We collected data describing the participants' behavior, objective performance and subjective perception. The courses were available to students between 7.4.2026 and 24.4.2026.

8.2 Data collection

Questionnaires

We collected data from 2 questionnaires - 1 taken before and 1 after filling out the courses. The answers were anonymized, and it is not possible to trace an answer to a specific user. We only knew which course variant the answer belongs to.

Unless stated otherwise, the questions had fixed answer selection on a scale from 0-10,

where 0 means strongly disagree and 10 means strongly agree.

The first questionnaire contained questions trying to assess the students' baseline attitudes and self-reported skill level:

- How would you rate your UML skills? (Likert 0-10)
- How interested are you in software modeling? (Likert 0-10)
- What is your attitude towards playful study materials? / What is your attitude towards competitive activities? (Likert 0-10)

The second questionnaire had significantly more questions, but they were still aimed at the students' attitudes towards e-learning, their satisfaction with the experience and self-reported skill level, including a number of open-ended questions. Full questionnaire responses are provided in Appendix A.

- How would you rate your UML skills? (Likert 0-10)
- I would like to encounter more e-learning materials during my studies at FIIT STU. (Likert 0-10)
- This course helped me develop a better understanding of the UML class diagram. (Likert 0-10)
- I would voluntarily take a similar course. (Likert 0-10)
- The course kept me engaged. (Likert 0-10)

Reports

We collected data regarding the students' objective performance. Names and email addresses were omitted from the reports, but we did keep the user identifiers. There are 2 variants of each mentioned report - one for the competitive course version and one for the playful course. That means though no feature describes which course a data point belongs to, the data are separated by course implicitly. The only exception is `Average_score_per_quiz_per_course`, which contain `courseid` as a feature.

Each question is categorized using tags, allowing us to examine the scores across various topics. We have average scores for each exercise and the overall number of attempts at them. This data was aggregated across all users divided by course variant.

The introductory and final assessment quizzes contain very similar exercises. That makes the difference between their relative scores a reasonable measure of progress of individual students. This data were collected per individual.

Similarly, we examined the performance in the first half of the courses' quizzes compared to the second half, and the students' final grades. However, some of the reports ended up unused in the analysis due to low interpretability or low data quality.

- Most difficult question (describes individual exercises)
 - `question_name` - name of the exercise

- quiz_name - name of the quiz in which the exercise is located
- tags - categories of the exercise
- attempts - total number of attempts at the exercise across all users in the course variant
- avg_score_percent - average score in the exercise normalized to a scale of 0-1
- variability - standard deviation of the score in the exercise
- difficulty_label - derived feature, describes an exercise's difficulty based on avg_score_percent
- Average_score_per_quiz_per_course
 - course_id - identifier of the course variant
 - quiz_name - name of the quiz
 - avg_score_percent - average score in the quiz across all users in the course variant
- Final grade across all users
 - finalgrade - the final grade, calculated as a sum of points gained across quizzes in the course. Quizzes had different weights - the final assessment quiz contributed the most significantly
- First half VS second half
 - first_group_score - score in the first 3 quizzes (expressed in percents)
 - second_group_score - score in the last 2 quizzes (expressed in percents)
 - improvement - calculated as second_group_score - first_group_score
- Intro vs final quiz improvement
 - intro_score - normalized score in the Introductory assessment quiz on scale 0-1
 - final_score - normalized score in the Final assessment quiz on scale 0-1
 - improvement - calculated as final_score - intro_score

Full reports are provided in Appendix A.

Moodle logs

Moodle stores data about user behavior, such as a user's activity start and end times, interactions with the system, or number of attempts for activities. We gain insights into how many times they viewed theory sections and how many times they interacted with quizzes (most of those interactions are reattempts at exercises). We collected this data for each user separately.

Measuring time taken to go through the course is unreliable, since the material is self-paced and students could take breaks in between course activities. For this reason, we approximated the time spent by only including quiz attempts. The data are still noisy, because the students could still take breaks in the middle of a quiz. But that is less likely

and left us with only a handful of outliers.

The most important features of the resulting datasets were:

- Interactions
 - interaction_type - name of the interaction type (theory/quiz/feedback)
 - activity_name - name of the activity (Introductory assessment quiz, Classes and objects theory page...)
 - interactions - number of interactions with an activity
- activity_time_sums
 - total_time_seconds - total time spent on quizzes in seconds
 - total_attempts - number of attempts at quizzes in total (5 at minimum)

Full reports are provided in Appendix A.

8.3 Data preparation

Questionnaires

To compare the introductory questionnaire and the final questionnaire, we created a composite index (design_index) in the feedback questionnaire by calculating the average responses of the following questions:

Playful variant: (What is your attitude towards playful study materials?)

- I would enjoy if other study materials were more playful as well
- The playful design (memes, badges, language) motivated me
- I enjoyed the playful narration in the theory section

Competitive variant: (What is your attitude towards competitive activities?)

- I enjoyed the aggressive narration in the theory section
- I would enjoy if I were to compete with my classmates more often
- Seeing my performance compared to others motivated me

We created another index (satisfaction_index) that numerically expresses the usefulness of creating similar e-learning materials. It was calculated as the average response to the following questions:

- I enjoyed the format of the course
- I would voluntarily take a similar course.
- The course kept me engaged.
- This e-learning material added value to the university course PSI.
- I would like to encounter more e-learning materials during my studies at FIIT STU.

We substituted responses to the question “Was any of the topics of the course insufficiently explained or hard to understand?” with the number of topics the student

marked.

Reports and logs

We combined the logs and reports from the previous section into a single table for better clarity and processing. This was only applied to reports that contained data for individual students - data about exercises, categories and questionnaire answers were not included.

Time taken to complete quizzes in the course was capped at 135 minutes. The reason is that students could leave a quiz open and leave, resulting in course attempts spanning even days. We only included time spent on quiz attempts, as total time spent on the courses was too blurred by students' breaks to be interpreted.

The resulting dataset contained features describing each student's performance, behavior and derived metrics. The resulting features were:

- id - user id for the participants' identification
- finalgrade - final grade calculated from weighted quiz results
- intro_score - score in the introductory assessment quiz
- final_score - score in the final assessment quiz
- improvement - final_score - intro_score
- duration_mins - time taken completing quizzes
- duration_cat - time taken completing quizzes expressed divided into 4 categories
- total_attempts - total number of attempts at quizzes
- quiz - number of interactions with quizzes
- theory - number of interactions with the theory pages
- theory/quiz interactions ratio
- extra_theory - number of theory pages subtracted from all theory interactions
- extra_theory/quiz interactions ratio
- extra_attempts - attempts at quizzes beyond the mandatory one attempt per quiz
- extra_theory_per_minute - extra_theory/duration_mins
- course - P or C, indicating which course the data point belongs to

Exercises

The data needed very little cleaning. The most difficult exercises made the categories they belonged to appear more difficult than they were. We attribute this to exercise design rather than insufficient understanding of the concepts. The exercises were removed from their respective categories, but kept for overall aggregate measurements.

Additional dataset was created that calculates the average performance across categories per course type.

8.4 Performance analysis

The questionnaires show lower self-reported UML competence in the Playful variant (PV) variant before taking the course. Self-reported competence equalizes after the students have completed the courses.

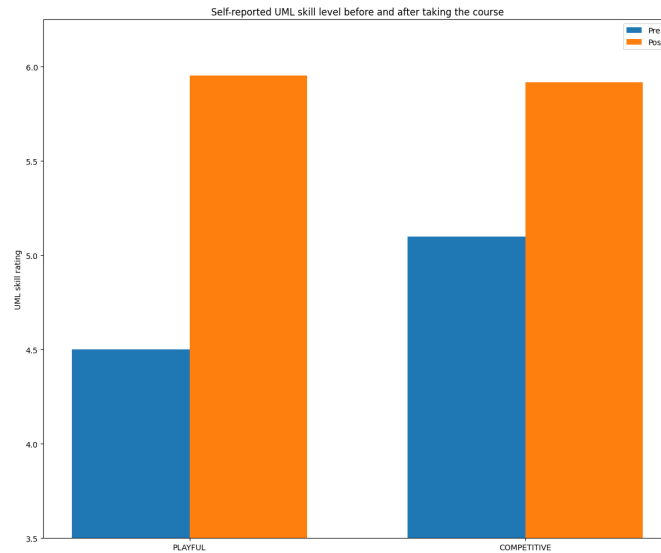


Figure 8.1: Self-reported skill in UML before and after taking the course

Despite the self-reported competence equalizing, the variance of the metric decreased in the PV, and increased in the Competitive variant (CV). That indicates the CV might have polarizing effects on the learners' perception.

Group	Phase	Mean	Median	Variance
Playful	Introduction	4.50	4.0	5.91
Playful	Feedback	5.95	6.0	5.15
Competitive	Introduction	5.10	5.0	2.73
Competitive	Feedback	5.92	6.5	5.72

Table 8.1: Self-reported UML skill ratings across course variants before (Introduction) and after (Feedback) the intervention.

The PV exhibits higher improvement rates among participants. The difference in improvement is not statistically significant (Mann–Whitney U, $p=0.21279$), a moderate effect size is observed (Cohen's $d=0.472$).

The Intro score is also lower in PV, while Final score is roughly equal in both variants. Along with a significant decrease in variance and standard deviation between Intro

score and Final score in PV, we can infer there is a ceiling to performance to which both groups converged. We should take note of students in PV starting with more diverse levels of skill.

Group	Metric	Mean	Median	Variance	Std
C	Improvement	1.47	2.48	113.95	10.67
C	Intro score	85.55	83.59	108.01	10.39
C	Final score	87.02	88.53	82.34	9.07
P	Improvement	6.81	6.12	137.68	11.73
P	Intro score	78.55	82.37	225.24	15.01
P	Final score	85.35	87.33	132.40	11.51

Table 8.2: Descriptive statistics of performance metrics for competitive (C) and playful (P) course variants.

We can see that students with lower Intro scores tended to improve more significantly, while high-scoring individuals even worsened in some cases.

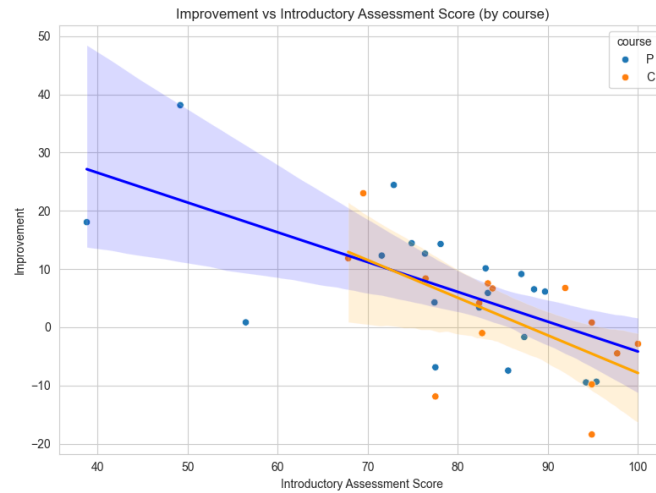


Figure 8.2: Improvement vs score in the introductory assessment quiz

The last figure shows performance across exercise categories. CV scores consistently higher in all categories except for one.

When comparing performance in individual exercises, PV only scores better in 5 out of 28. That is a noticeable difference, which might not be attributed to a slightly higher baseline knowledge alone.

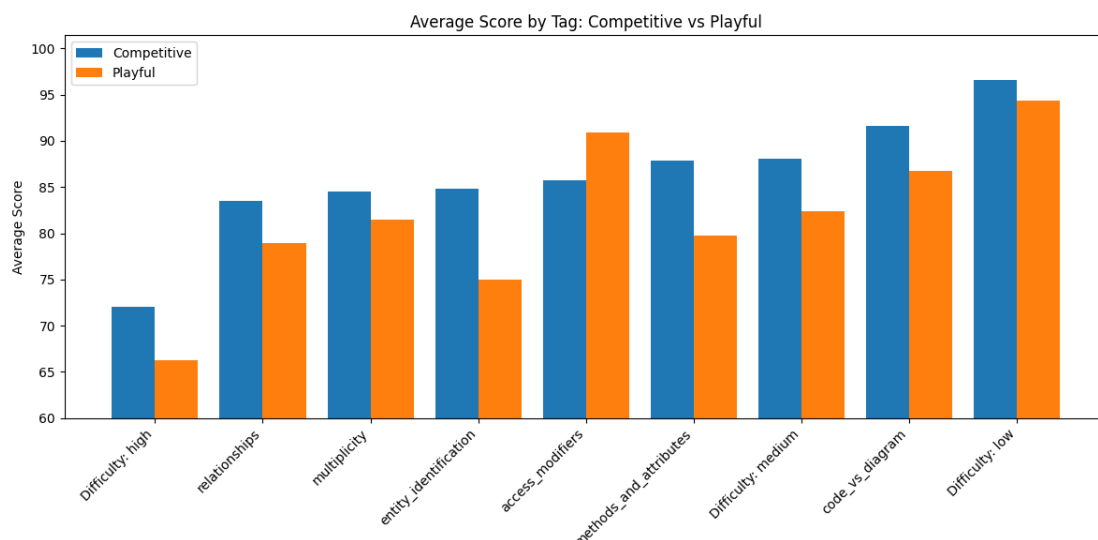


Figure 8.3: Average score by tag for Competitive and Playful variants

In performance across tags, we cannot observe any obvious patterns, as CV and PV did not struggle with the same topics. What can be noted is a more pronounced difference in `entity_identification` scores. Those exercises required more patience, since they involved reading a textual description. That may hint that participants in CV were less willing to rush exercises out of impatience.

For both groups the `code_vs_diagram` category was in the top 2 places score-wise (excluding the difficulty markers). That supports the findings of other researchers that students understand diagrams better when linked to code.

8.5 Behavior analysis

In the next table we can see the way the students interacted with the course. Though the differences are not strikingly high, we can observe a behavioral pattern; while PV interacted with theory more, CV engaged with quizzes more.

The number may appear small, but CV made 7 extra attempts at quizzes in total, while PV only 2 despite having more participants. After scaling, that is more than 500% increase.

Metric	C Mean	C Std	P Mean	P Std	Interpretation
Quiz interactions	115.57	17.38	110.90	11.66	Higher in C
Extra theory	12.43	6.57	16.71	10.47	Higher in P
Interaction ratio	0.11	0.06	0.15	0.10	Higher theory focus in P
Extra attempts	0.50	0.85	0.10	0.30	Higher retry behavior in C

Table 8.3: Simplified comparison of engagement behavior between Competitive (C) and Playful (P) variants.

Time is difficult to track, since in cases students took breaks with a quiz attempt open, the idle time is included in the calculation as well. However, another pattern can be seen in improvement rates plotted against time spent in the course. While PV shows consistent improvement rates across time categories (90-120 minutes contains a single instance, and we view it as an outlier). The final grade in PV is also rather consistent.

In the CV we can observe a different pattern. Most notably students who had high grades in the intro quiz achieved lower grades in the <60 mins category, while those in the 90-120 and 120+ categories did not experience a drop in performance. That displays a loss of interest and subsequent rushing in some students, which we did not observe in PV.

The increased improvement rates in CV in 60-90 mins category can be explained by the group having lower intro scores.

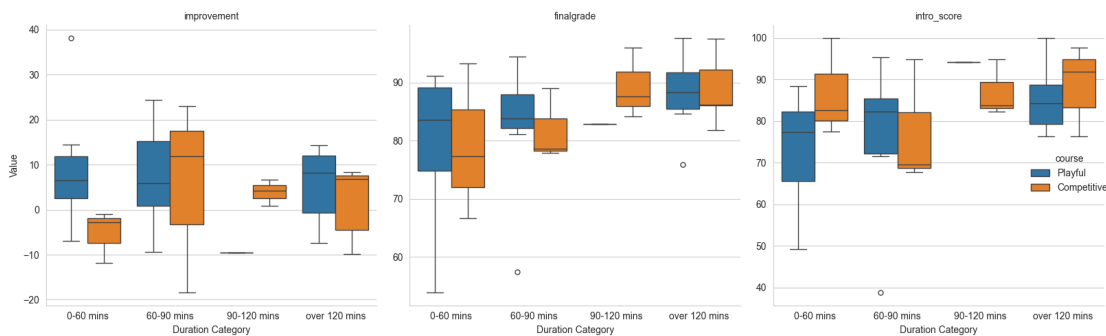


Figure 8.4: Improvement, final grade and intro score by time spent on the course

8.6 Engagement analysis

8.6.1 Textual responses

The feedback questionnaire contained 2 open-ended questions in each variant.

Textual responses - Competitive variant

Do you have any comments or suggestions for improvement regarding this course?

There were 2 answers in total. One student stated: “It was really educational.” Apart from that, we only received suggestions for improvement in the course design - a request for more clear requirements in cardinality exercises and a comment about multiple tries mechanics in exercises.

What effect did the slightly aggressive narration in the theory sections have on you?

Only one answer was provided: “Positive”.

Textual responses - Playful variant

The PV received significantly more responses that were both more elaborate and more enthusiastic. That is indicative of higher engagement rate and a better mood in the participants after having taken the course compared to CV.

Do you have any comments or suggestions for improvement regarding this course?

Students made multiple suggestions for improvement, such as adding more hints after incorrect responses, improving navigation in the course, or adding a summary page for all the concepts. There was a complaint that some questions did not have fully deterministic answers.

The students expressed their sentiments about the course with quotes:

- “This course was absolute cinema”
- “I really enjoyed it, i could see this be a part of subjects curriculum and learn from such mini tests and quizzes (funny references must be included)”
- “It was more fun than I expected, if I actually cared about software modeling, I would enjoy learning it via this site :3”
- “It is quite good”
- “It is great! I liked it! It contains all the basics of class diagram modeling. Also it is pretty funny, thank you for memes”

What effect did the playful narration in the theory sections have on you?

The answers here were predominantly positive.

- “My motivation and enthusiasm would be almost zero if not for the playful narration”
- “Thanks to the playful narration, the theory felt more human-like and less as something I should be afraid of.”
- “It made me not want to kill myself, but I mostly just skipped those”

- “the story narration and real life examples were very good, however the memes did not do that much for me: /”
- “enjoyed it.”

8.6.2 Numeric responses

In many features, we can further observe the polarizing effects of the competitive design: while in PV the responses tend to be more grouped around the center, CV have more spread out distributions with more extreme values.

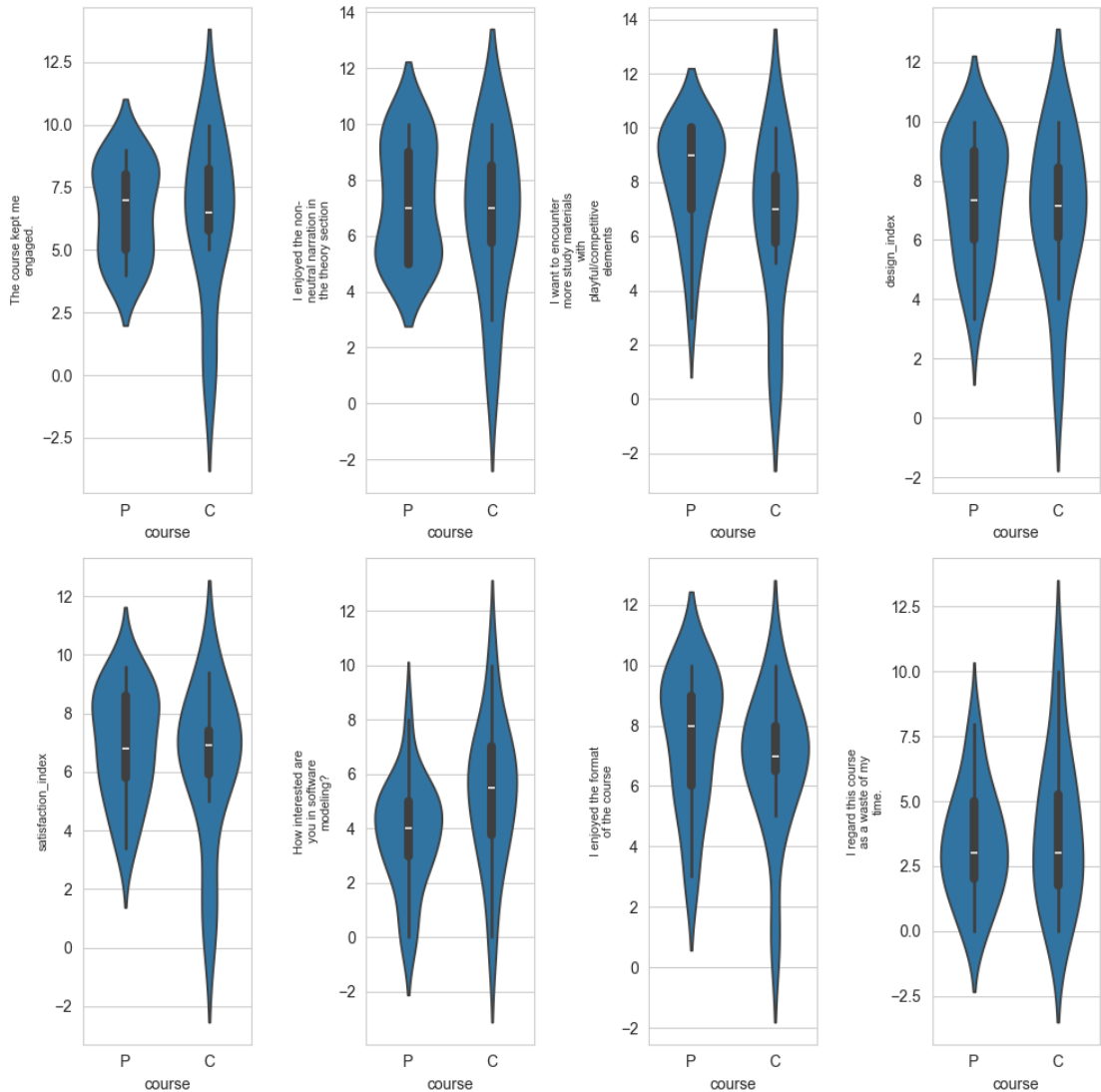


Figure 8.5: Distributions of subjective metrics in the feedback questionnaire

In most features, PV displays higher engagement and satisfaction values than CV. That corresponds with observations from open-ended questions in the feedback questionnaire (see Section 8.6.1).

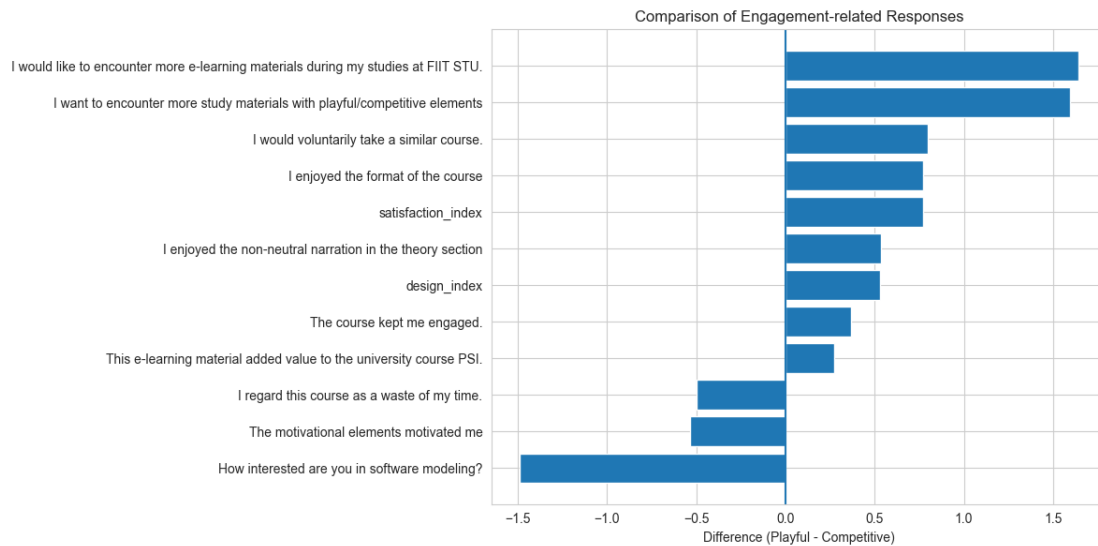


Figure 8.6: Difference of means (PV-CV) in engagement and general satisfaction metrics in the feedback questionnaire

Despite the higher engagement rates and general satisfaction of participants in PV, there is one interesting drawback of the course - while in CV the interest in software modeling remained the same before and after the course was taken, PV shows a decrease in interest. There are several possible explanations for the observation. Students take the topic less seriously when presented in a playful tone, extrinsic rewards may shift motivation away from intrinsic interest. However, the difference is not statistically significant (Mann-Whitney U, $p=0.420229$, Cohen's $d=-0.316$) and should not be interpreted as a robust effect.

Preference for both competitive and playful elements dropped slightly between the start and end of the courses. The decrease is not statistically significant in either variant (Mann-Whitney U, PV - $p=0.295627$, CV - $p=0.315938$) and can be attributed to discrepancy between what the students imagined as playful or competitive and the content of the courses, and to mental fatigue from the tasks.

Notably, the difference between the drops roughly corresponds to the difference in answers to the question "The course kept me engaged" in the feedback questionnaire (see Figure 8.6), suggesting a potential relationship between engagement and the drop in preference.

The answer to the questions “What is your attitude towards competitive activities?” and “What is your attitude towards playful study materials?” are not in the feedback questionnaire directly - the `design_index` value was used instead (see Section 8.3).

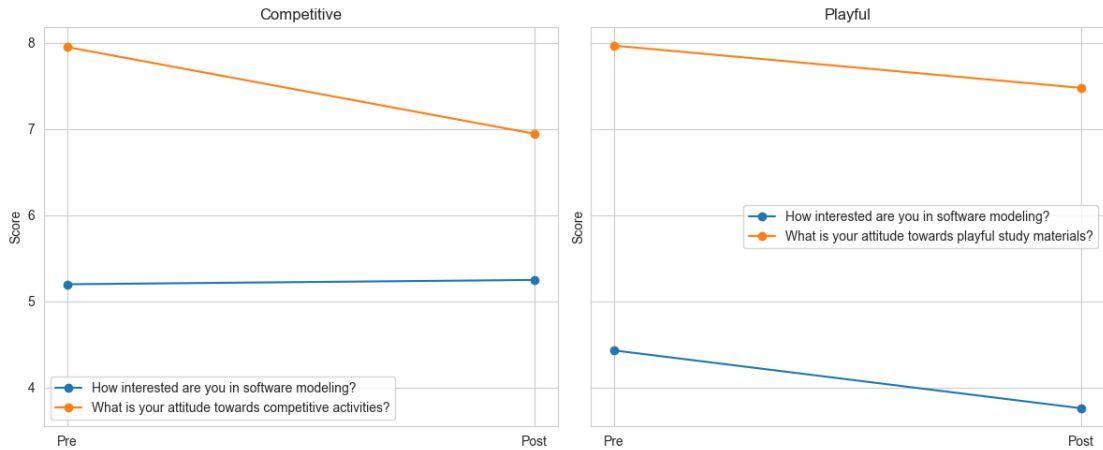


Figure 8.7: Perceptions of student before and after taking the courses

In the following correlation matrices we can observe an interesting pattern. In PV, there is weak correlation between interest in software modeling and perceived skill level ($r=0.17$). Interest in playful study materials is largely independent of the other metrics as well.

In contrast, CV group shows a strong relationship between interest in software modeling and self-reported UML skill ($r=0.83$). What is noticeable is that enthusiasm for competitive activities is more strongly related ($r=0.72$) to interest in software modeling rather than perceived skill ($r=0.47$). This indicates a more tightly coupled response structure, where students who report higher interest in software modeling also tend to report higher perceived competence and more positive attitudes toward competition.

This coupled with CV having lower participation rate and higher baseline skill level suggests that students were more likely to be put off by CV - unless already interested and competent. PV did not present a barrier for beginner or disinterested students.

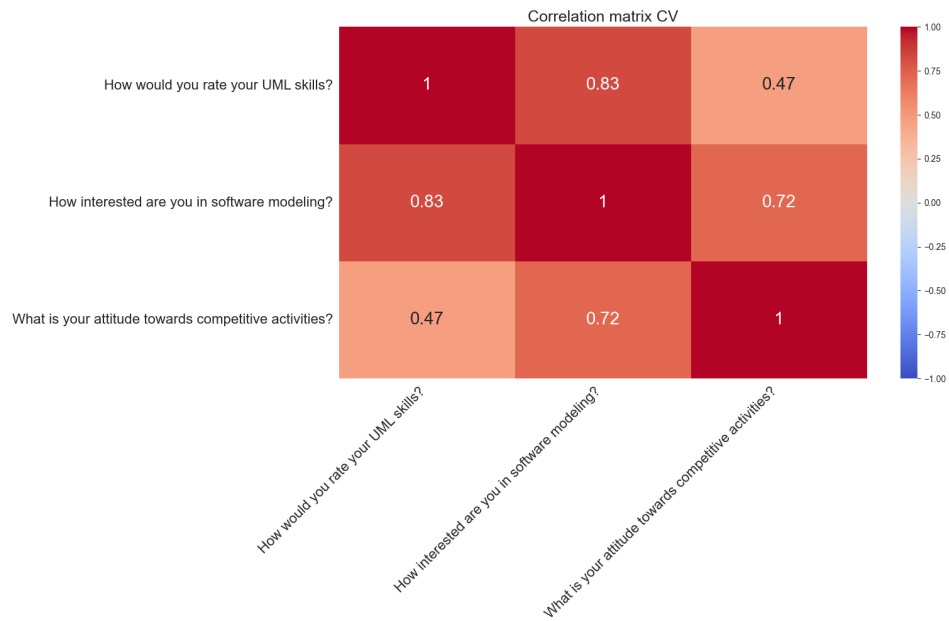


Figure 8.8: Correlation matrix for feedback questionnaire - CV

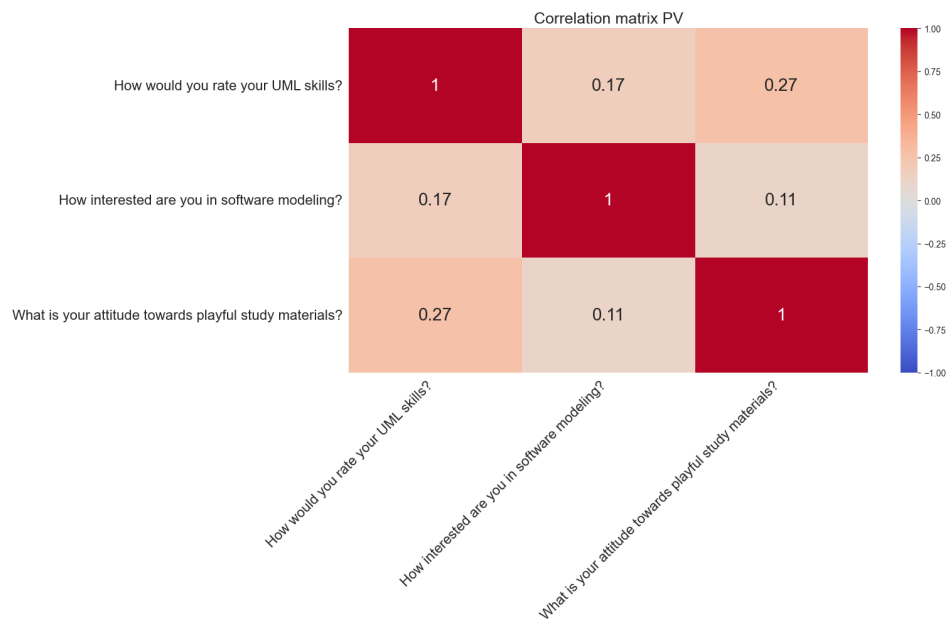


Figure 8.9: Correlation matrix for feedback questionnaire - PV

8.7 Prospects for future e-learning projects on FIIT STU

A part of the evaluation is determining the students' interest in e-learning materials at FIIT STU and assessing the usefulness of conducting further research in the topic.

The feedback questionnaire displays an enthusiastic approach towards e-learning materials, especially in the PV group. That is further reinforced by textual responses (see Section 8.6.1) to its open-ended questions.

The PV group expressed more interest in e-learning, as well as a very high (8+) interest in encountering more non-neutral study materials. Average general satisfaction with the courses is also higher in PV than in CV, though high in the combined measurements as well.

Both groups agreed that the courses added value to the subject they were asked to fill out the course in, and that it helped them improve their understanding of the UML class diagram in roughly equal measure.

Responses to "I would voluntarily take a similar course" show less conviction - the students might only take such courses if offered extra credits for doing so or if it were mandatory. Another interpretation of the question results are that they would voluntarily take a course - if it were better designed.

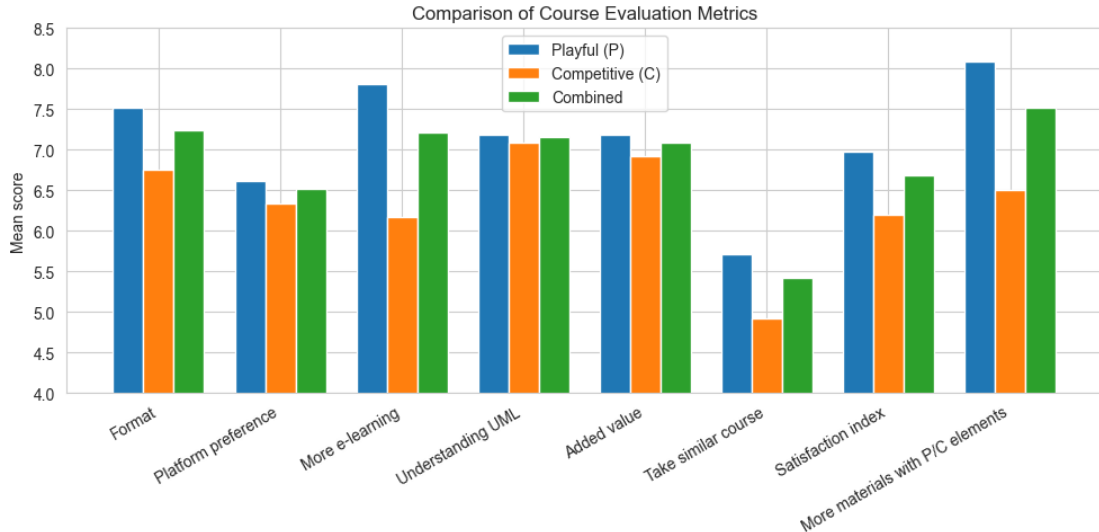


Figure 8.10: Students' opinions on e-learning at FIIT STU

Full metric names in respective order:

- I enjoyed the format of the course.

- If there are other courses, I would not mind them being made using the same platform as this one (Moodle).
- I would like to encounter more e-learning materials during my studies at FIIT STU.
- This course helped me develop a better understanding of the UML class diagram.
- This e-learning material added value to the university course PSI.
- I would voluntarily take a similar course.
- satisfaction_index (see Section 8.3 for calculation)
- I want to encounter more study materials with playful/competitive elements

8.8 Limitations

Our experiment faces several limitations that might weaken our claims.

Data

The questionnaire responses were anonymized, not allowing us to connect students' behavior and performance to their attitudes.

The groups in our experiment were not of same size, with some participants dropping out during the course (at the end of the course: PV - 21, CV - 14). That might affect findings from comparing the introductory and feedback questionnaires. The relatively small sample size reduces the statistical power of our findings.

On top of that, the participants did not have the same baseline knowledge across both courses, making the interpretation of collected data more complicated.

Lastly, it should be noted that the difference in overall satisfaction with the course as well as interest in e-learning in general might be at least partially attributed to a potentially better design of PV compared to CV. However, there is no way to verify the assumption without further experiments.

Course design

One hour long course does not allow for implementation of some concepts, such as techniques related to gradual increase in difficulty, meaningful progressions, or advanced adaptive learning techniques. Additionally, an activity that spans a longer time frame might display more significant differences in observed groups.

Another drawback was the combination of the absence of an automated diagram assessment tool and the unavailability of teacher intervention. This led us to substitute direct modeling exercises with alternatives that were easier to correct automatically.

8.9 Summary

The data show that objective performance is higher in CV, while improvement rates are higher in PV. This is explained mainly by the fact that CV has a higher baseline skill in UML, which also reduces the potential for improvement among its participants.

The results show consistently higher scores across activities in CV. This may be related not only to higher starting point, but also to behavior patterns. Students in CV engaged with and retried quizzes more often than students in PV.

On the other hand, students in PV engaged more with the theoretical content. This indicates differences in interaction patterns between the groups - an exploratory approach in PV and a goal-oriented approach in CV.

The textual responses and questionnaire results indicate better overall reception and higher enthusiasm than CV across most subjective metrics. However, PV also shows a decrease in interest in software modeling after the end of the course, suggesting a potential risk of shifting motivation away from intrinsic interest if not designed carefully.

Responses in CV are more variable, indicating a polarizing effect of the design. PV responses are more consistent, suggesting greater accessibility.

Overall, competitive design appears more suitable for already skilled and interested students, while playful design is more accessible to a broader audience.

Students expressed general interest in e-learning materials and their use at FIIT STU. The reception of this course suggests further experiments on the topic of e-learning at FIIT STU are justified.

RQ1: Does competitive course design lead to higher learning performance compared to playful design?

Answer to RQ1: Participants in CV achieved consistently higher scores across exercises, categories and assessments. However, participants in PV achieved higher improvement rates, which can be largely attributed to lower baseline performance and more room for improvement. Although results suggest a slight performance advantage in CV, both approaches lead to comparable learning outcomes overall.

RQ2: How do playful and competitive course designs influence students' behavioral engagement?

Answer to RQ2: The designs led to different behavioral patterns. Participants in PV interacted more with the theoretical content, while participants in CV had more quiz

interactions and attempts. This suggests a tendency towards more exploratory behavior in PV, and a more goal-oriented attitude in CV.

RQ3: How do students evaluate playful and competitive course designs in terms of emotional engagement, satisfaction, and motivation?

Answer to RQ3: Participants in PV expressed higher enthusiasm, satisfaction and engagement in nearly all metrics as well as textual responses.

However, they also showed a drop in interest in software modeling itself between the introduction and feedback questionnaires. This may indicate a shift in motivation or a difference in how the topic is perceived when presented in a playful manner, although the effect is not statistically significant.

Responses in CV exhibited higher variability, indicating that competition may have a polarizing effect on the students' perceptions. PV had more consistent responses and proved to be more accessible in general.

RQ4: What are students' perceptions of the usefulness and future potential of e-learning materials at FIIT STU?

Answer to RQ4: Students perceived the course as a valuable addition to the software modeling subject and expressed interest in encountering more e-learning materials during their studies at FIIT STU. Higher general interest was expressed regarding playful materials than competitive ones.

Chapter 9

Conclusion

9.1 Summary

Software modeling and UML are important parts of software development. They are included in universities' curriculum not only because they are used in practice, but also because they help students gain a broader perspective on programming and software development - they teach them to plan before writing code and consider the abstract and business aspects of systems.

However, it is common for students and professionals alike to claim knowledge of UML, while having only limited or incorrect actual skills in the area.

Traditional education of UML faces several challenges such as low amount of feedback, late feedback, or a lack of space for personal attention from teachers.

A possible solution to the problem lies in e-learning, which has the potential to improve students' overall learning performance, engagement, and information retention across many domains. It achieves this by allowing for features such as instant feedback, personalized content recommendations, or interactive exercises where students can learn by doing immediately instead of listening to lectures.

Education of software modeling is no different. Past research has demonstrated promising outcomes by introducing gamified courses, automated diagram assessment tools, exercise recommendation systems and others.

We aimed our research at creating an e-learning course about the UML class diagram. Our research goals were twofold. Firstly, the use of e-learning at FIIT STU is limited, and we wanted to prepare the ground for further research in the area. Secondly, most research aims at comparing traditional education and e-learning rather than various e-

learning approaches. This work contributes to the area by comparing 2 common gamification practices - competition and playfulness. For the experiment, we created 2 variants of the same course applying the 2 designs.

The testing of our course prototypes showed differences in behavioral and emotional engagement related to the 2 designs. However, differences in objective learning outcomes were inconclusive and mostly attributable to differences in starting skill levels of the studied groups.

Participants in the playful variant exhibited a tendency towards a more explorative approach to the materials, while the competitive variant may promote a more goal-oriented approach directed at measurable outcomes.

Responses to the playful design demonstrated higher interest in encountering more e-learning materials, engagement and overall satisfaction and enthusiasm compared to participants in the competitive design.

Students in the competitive course exhibited more varied responses to the satisfaction metrics, indicating a polarizing effect of competition.

The overall reception of the courses was positive and interest in encountering more e-learning materials at FIIT STU was expressed. These findings suggest that creation of more e-learning content at FIIT STU is justified and desirable.

9.2 Future Work

Based on our experiment results and findings from analysis, we recommend conducting further research on the application of e-learning on software modeling - and potentially other areas of software engineering as well.

Future experiments may extend this work in multiple ways. Tools for enhancing educational processes in software modeling could be integrated into our course - most notably an automated diagram assessment tool, or an exercise generator. The course can be extended to include multiple diagram types, or even new courses aiming other areas of software engineering can be created.

Adaptive and personalized e-learning practices could be explored to address the varying baseline skill levels observed in our experiment. Other course designs can be implemented and compared for future reference.

Regarding competitive practices, we suggest implementing them in a manner that allows students to make use of the materials without participating in the competition. In-person, group activities might be more suitable for competitive activities.

Playful materials seem accessible to most audiences. Further research may focus on aspects of it not explored in this work, and make an effort to ensure it does not distract students from their learning objectives.

Experiments could span over longer periods of time so that compound effects of the studied approaches could be demonstrated more clearly.

Resumé

Úvod

UML je široko používaný nástroj v oblasti softvérového modelovania, ktorý umožňuje opisovať systémy na abstraktnej úrovni z rôznych perspektív. Schopnosť porozumieť mu a správne ho používať je dôležitá pre študentov aj profesionálov, no v praxi sa často ukazuje, že tieto znalosti sú len povrchné alebo nepresné.

Tradičné vzdelávanie často neposkytuje dostatok priestoru na individuálny prístup a priebežnú spätnú väzbu, čo vedie k nižšej úrovni praktických zručností. Jedným z riešení je e-learning, ktorý umožňuje využívať interaktívne materiály, okamžitú spätnú väzbu a praktické cvičenia. Zároveň podporuje angažovanosť študentov prostredníctvom prvkov gamifikácie, ako sú súťaž alebo zábavné prvky.

Diagram tried patrí medzi najdôležitejšie UML diagramy, no študenti s ním majú napriek tomu často problémy. Cieľom tejto práce je preto navrhnuť e-learningový kurz zameraný na jeho výučbu ako doplnok k univerzitnému predmetu.

Kurz využíva moderné e-learningové prístupy a je implementovaný v dvoch variantoch – zábavnom a kompetitívnom. Tie boli experimentálne overené na študentoch FIIT STU s cieľom porovnať ich vplyv na výsledky, správanie a subjektívne vnímanie.

UML

UML je štandardizovaný vizuálny modelovací jazyk, ktorého prvá verzia bola publikovaná v roku 1997 a neskôr rozšírená na UML 2.0 v roku 2005 s cieľom zjednotiť rôzne prístupy k modelovaniu softvérových systémov. Najnovšou verziou je UML 2.5.1 z roku 2017. Jazyk slúži na špecifikáciu, dokumentáciu a vizualizáciu softvérových systémov prostredníctvom diagramov.

UML definuje viacero typov diagramov, ktoré možno rozdeliť na štrukturálne a behaviorálne. Štrukturálne diagramy (napr. diagram tried, objektov, komponentov alebo nasadenia) zobrazujú statickú štruktúru systému pomocou tried, atribútov, operácií a vzťahov. Behaviorálne diagramy (napr. sekvenčný, aktivitný alebo stavový diagram) sa

zameriavajú na dynamiku systému, teda interakcie medzi objektmi a zmeny ich stavov počas vykonávania.

Diagram tried

Diagram tried patrí medzi najdôležitejšie UML diagramy a slúži na opis statickej štruktúry systému, jeho prvkov a vzťahov medzi nimi. Využíva sa naprieč viacerými fázami vývoja softvéru – od analýzy až po implementáciu – pričom môže pracovať s rôznymi úrovňami abstrakcie. Okrem dokumentácie systému môže slúžiť aj ako podklad pre generovanie kódu.

Základnými prvkami diagramu tried sú triedy, ich atribúty, operácie a vzťahy.

Trieda predstavuje šablónu pre objekty, zatiaľ čo objekt je jej konkrétna inštancia.

Atribúty definujú vlastnosti triedy vrátane dátových typov a viditeľnosti.

Operácie reprezentujú správanie objektov a zahŕňajú názov, parametre a návratovú hodnotu.

Viditeľnosť určuje prístup k prvkom (napr. *public*, *private*) a modifikátory ako *abstract*, *static* alebo *final* bližšie špecifikujú ich správanie.

Diagram tried podporuje viacero typov vzťahov medzi triedami. Asociácia predstavuje všeobecné prepojenie medzi triedami.

- Agregácia a kompozícia vyjadrujú vzťah celok–časť, pričom kompozícia je silnejšia a viaže životný cyklus častí na celok
- Dedičnosť umožňuje vytvárať hierarchie tried a zdieľať vlastnosti medzi nimi
- Závislosť vyjadruje dočasné použitie jednej triedy druhou a realizácia popisuje implementáciu rozhraní

Dôležitou súčasťou vzťahov je aj kardinalita, ktorá určuje počet inštancií zapojených do vzťahu.

E-learning

E-learning je digitálny prístup k vzdelávaniu, ktorý umožňuje flexibilné a samostatné štúdium. Oproti tradičnej výučbe prináša vyššiu dostupnosť a individualizáciu, no vyžaduje väčšiu disciplínu a prináša nižšiu mieru sociálnej interakcie.

Výskumy ukazujú, že e-learning má potenciál zlepšiť motiváciu aj výsledky študentov.

Medzi hlavné techniky patria gamifikácia, mikro-učenie, adaptívne a self-directed učenie, zmiešané učenie a sociálne učenie. Ich kombinácia vedie k lepšej angažovanosti a efektívnosti učenia.

Využitie e-learningu pri vyučovaní diagramu tried

Výskum v oblasti výučby UML diagramu tried sa dlhodobo zameriava na zlepšenie motivácie študentov a efektivity učenia pomocou nástrojov ako sú programy na automatizované opravovanie diagramov, alebo e-learningových prístupov, najmä gamifikácie, mikro-učenia a kolaboratívneho učenia.

Gamifikácia patrí medzi najčastejšie skúmané prístupy. Existujúce riešenia využívajú mechaniky ako body, úrovne, skúsenostné systémy, odznaky, rebríčky či prispôsobiteľné avatary. Cieľom je zvýšiť zapojenie študentov prostredníctvom motivácie, súťaženia a okamžitej vizuálnej spätnej väzby.

Výsledky štúdií vo všeobecnosti ukazujú pozitívny vplyv na motiváciu aj výkon, zároveň však upozorňujú, že gamifikácia musí byť navrhnutá opatrne, pretože príliš komplexné alebo povrchové mechaniky môžu spočiatku znižovať výkon, kým si na ne študenti zvyknú.

Mikro-učenie sa využíva najmä rozdelením učiva do menších tematických celkov. Tento prístup znižuje kognitívne zaťaženie a zlepšuje pochopenie najmä v počiatočných fázach učenia. Na druhej strane môže byť menej vhodný pri komplexnejších témach, kde je potrebné prepájať väčšie množstvo konceptov.

Kolaboratívne a sociálne učenie, napríklad diskusie, zdieľanie riešení či analýza prác iných študentov, preukázalo pozitívny vplyv na porozumenie aj motiváciu. Tieto prístupy podporujú reflexiu a umožňujú študentom vidieť alternatívne riešenia.

Významnú časť výskumu tvoria aj automatizované nástroje na hodnotenie UML diagramov. Ich cieľom je znížiť záťaž učiteľov automatickým vyhodnocovaním riešení a poskytovaním spätnej väzby porovnaním so vzorovým riešením.

Hoci sú tieto systémy efektívne z pohľadu času, narážajú na problém viacerých správnych riešení a obmedzenú schopnosť poskytovať detailnú spätnú väzbu. Objavujú sa aj prístupy založené na strojovom učení, tie však zatiaľ neposkytujú dostatočne interpretovaliteľné výsledky.

Ďalšie práce skúmajú nástroje na prevod diagramov do kódu a odporúčacie systémy, ktoré pomáhajú prepájať abstraktné UML koncepty s implementáciou a usmerňujú študentov pri učení.

Celkovo literatúra naznačuje, že najefektívnejšie prístupy kombinujú štruktúrované delenie obsahu, okamžitú spätnú väzbu a motivačné prvky ako gamifikácia a sociálne porovnávanie.

V literatúre ale môžeme pozorovať nedostatok výskumu zameraného na porovnávanie vplyvu rôznych kombinácií týchto prvkov, čo vytvára priestor pre ďalší výskum.

Návrh riešenia

Návrh kurzu vychádzal z obmedzení rozsahu a cieľov práce. Kurz bol koncipovaný ako krátky, približne hodinový doplnok k existujúcemu predmetu, zameraný výlučne na diagram tried, ktorý je jedným z najpoužívanějších a pre študentov najzrozumiteľnejších UML diagramov.

Riešenie bolo navrhnuté na existujúcej LMS platforme, ktorá umožňuje jednoduchú implementáciu obsahu, zaznamenávanie výsledkov a podporu interaktívnych cvičení.

Štruktúra kurzu pozostáva zo štyroch častí: vstupný test, teoretická časť, praktické cvičenia a záverečný test doplnený o dotazník.

Teória je organizovaná postupne, po malých celkoch, pričom jednotlivé koncepty sú vysvetľované na príklade tvorby diagramu.

Cvičenia nahrádzajú priame modelovanie a sú navrhnuté tak, aby simulovali jeho aspekty a poskytovali okamžitú spätnú väzbu. Cvičenia boli takto navrhnuté pre absenciu nástroja na automatizované vyhodnocovanie diagramov a nutnosti mať plne automatizovanú spätnú väzbu.

Kurz využíva viacero e-learningových techník, najmä mikro-učenie, gamifikáciu, adaptívne prvky, self-paced a blended learning. Rozlíšenie medzi variantmi je založené na aplikácii zábavných a kompetitívnych prvkov, ako sú odmeny, odznaky, porovnávanie výkonu alebo emotívne podfarbená narácia kurzu.

Vyhodnotenie riešenia pozostáva zo subjektívnej časti (dotazníky), objektívnej časti (vstupný a výstupný test) a analýzy správania študentov (interakcie, čas, aktivita v systéme).

Funkčné požiadavky

- Automatické vyhodnocovanie cvičení a testov s okamžitou spätnou väzbou
- Podpora e-learningových techník
 - Podpora gamifikácie (vizuálna spätná väzba, indikátory náročnosti, odznaky)
 - Podpora mikro-učenia
 - Podpora adaptívneho učenia (odporúčanie obsahu na základe výsledkov študenta)
 - Podpora sociálneho učenia (zobrazenie anonymizovaných výsledkov ostatných študentov)
- Podpora interaktívnych cvičení a študijných materiálov:
 - UML kódové ukážky prepojené s diagramami
 - príklady diagramov s vysvetlením návrhových rozhodnutí
 - drag-and-drop cvičenia
 - párovacie a výberové úlohy

- teoretické otázky
- Zber anonymnej spätnej väzby od študentov
- Testovanie vedomostí študentov
- Monitorovanie správania študentov v systéme

Nefunkčné požiadavky

- Jednoduchosť použitia a učenia sa systému
- Open-source alebo bezplatné riešenie
- Ukladanie a správa dát o študentoch
- Škálovateľnosť a udržiavateľnosť pre budúce rozšírenia
- Jednoduchá rozšíriteľnosť o nové funkcie a obsah
- Obmedzený prístup len pre študentov a učiteľov FIIT STU
- Prístupnosť obsahu kedykoľvek a odkiaľkoľvek

Opis riešenia - obsah kurzov

Na implementáciu kurzu sme zvolili LMS systém Moodle, ktorý spĺňa väčšinu našich funkčných aj nefunkčných požiadaviek. Ide o open-source riešenie, ktoré je možné nasadiť na vlastnú infraštruktúru, čím získavame kontrolu nad dostupnými zdrojmi a možnosť škálovania.

Nevýhodou je absencia natívnej podpory pre tvorbu UML diagramov. Tento nedostatok sme riešili využitím alternatívnych typov úloh, najmä drag-and-drop cvičení simulujúcich proces modelovania.

Teoretická časť kurzu je realizovaná pomocou zdroja *Book*, ktorý umožňuje štruktúrovať obsah do kapitol a podkapitol podobne ako v klasickej učebnici. Študent tak má prehľad o svojom postupe a môže sa k obsahu jednoducho vracať.

Výklad je postavený na postupnom vytváraní ukážkového diagramu tried, pričom jednotlivé koncepty sú zavádzané postupne a v kontexte praktického príkladu (Fig 7.2).

Prvá časť sa zameriava na základné pojmy, ako sú triedy, objekty, atribúty, metódy či modifikátory prístupu. Druhá časť sa venuje vzťahom medzi triedami a ich vlastnosťami.

Po každej časti nasleduje krátky kvíz na overenie porozumenia.

Cvičenia tvoria kľúčovú časť kurzu. Vytvorili sme spolu 29 úloh rôznych typov, ktoré simulujú reálne modelovacie situácie.

Študenti napríklad identifikujú entity zo zadania, určujú typy vzťahov, hľadajú chyby v diagramoch alebo priradujú kód k diagramom. Významnú časť tvoria aj úlohy, kde študenti skladajú diagram pomocou drag-and-drop komponentov (Fig 7.5).

Pri nesprávnych odpovediach systém poskytuje spätnú väzbu, vysvetlenia alebo odkazy na relevantnú teóriu.

Na testovanie vedomostí sme využili aktivitu *Quiz*, ktorá umožňuje kombinovať úlohy do celkov.

V priebežných častiach kurzu je dostupná okamžitá spätná väzba a možnosť opakovaných pokusov, pričom počet bodov sa pri opakovaní znižuje. Naopak, vstupný a záverečný test majú obmedzený počet pokusov a neposkytujú okamžitú spätnú väzbu.

Moodle zároveň poskytuje rozsiahle štatistické prehľady o výsledkoch študentov. Pre potreby experimentu sme vytvorili aj vlastné reporty pomocou pluginu *Configurable reports*, ktoré umožňujú detailnejšiu analýzu správania a výkonu študentov.

Niektoré z týchto reportov boli sprístupnené aj študentom, napríklad na porovnanie ich výkonu s ostatnými.

Zber subjektívnych dát bol realizovaný prostredníctvom aktivity *Feedback*. Tá umožňuje anonymné odpovede na otázky týkajúce sa spokojnosti, vnímania kurzu a celkovej skúsenosti študentov.

Tieto údaje predstavovali dôležitý vstup pre následnú analýzu experimentu.

Opis riešenia - e-learningové techniky a rozlíšenie variantov

V kurze sme implementovali viacero bežných e-learningových techník s cieľom zvýšiť efektivitu učenia a zapojenie študentov.

Využili sme princípy mikro-učenia rozdelením teórie na krátke, tematicky úzko zamerané celky, pričom medzi jednotlivé časti boli vložené cvičenia. Tým sa podporilo priebežné upevňovanie vedomostí.

Dôležitou súčasťou bola okamžitá spätná väzba. Po každej odpovedi študent získal informáciu o správnosti riešenia spolu s vysvetlením, prípadne odkazom na relevantnú teóriu. K dispozícii mal aj prehľad o vlastnom výkone.

Pre lepšiu orientáciu bola náročnosť úloh vizuálne označená.

Zábavný variant (PV) využíval gamifikačné prvky, ako sú odmeny za správne odpovede, odznaky za dosiahnutie míľnikov a neformálny, odľahčený štýl komunikácie v teoretických častiach.

Kompetitívny variant (CV) bol postavený na prvkoch súťaženia. Obsahoval rebríček najlepších študentov, porovnanie individuálneho výkonu s priemerom kurzu a zobrazenie priemerných výsledkov pri jednotlivých úlohách. Komunikácia v tomto variante bola asertívnejšia a dôraznejšia.

Oba prístupy predstavujú odlišné stratégie motivácie študentov, ktoré boli následne predmetom experimentálneho porovnania.

Nasadenie riešenia

Riešenie bolo nasadené na serveri poskytnutom FIIT STU počas trvania experimentu. Parametre servera boli nasledovné:

- Operačný systém: Ubuntu 22.04 LTS
- Počet virtuálnych CPU jadier: 2
- Veľkosť pamäte: 4 GB
- Veľkosť úložiska: 50 GB

Kapacita servera bola navrhnutá pre maximálne desiatky používateľov súčasne. Pre zabezpečenie prístupu bola využitá doména *fiiture.sk*, ktorú nám poskytol Ing. Juraj Vincúr, PhD. Registrácia nových používateľov bola zakázaná a účty boli vopred vytvorené a distribuované študentom, čím sa obmedzil prístup len na cieľovú skupinu.

Opis experimentu

Experiment bol realizovaný na vzorke študentov FIIT STU zapísaných na predmet softvérové modelovanie. Študenti boli náhodne rozdelení do dvoch variantov kurzu – zábavného (PV) a kompetitívneho (CV), aby sa predišlo skresleniu výberom podľa preferencií. Účasť bola dobrovoľná a motivovaná bodovým ohodnotením. Celkovo kurz dokončilo 35 študentov (PV: 21, CV: 14).

Dáta

Dáta (priložené v časti Appendix A) boli zbierané z viacerých zdrojov: dvoch dotazníkov (pred a po kurze), systémových reportov a logov z prostredia Moodle.

Dotazníky zachytávali sebahodnotené schopnosti, postoje k e-learningu a subjektívne vnímanie kurzu, pričom obsahovali aj otvorené otázky.

Výkonnostné dáta zahŕňali skóre v kvízoch, zlepšenie medzi vstupným a záverečným testom a výsledné hodnotenie.

Logy poskytovali informácie o správaní študentov, ako napríklad počet interakcií s teóriou a kvízmi či počet pokusov.

Na účely analýzy boli vytvorené agregované metriky, napríklad *design_index* (postoj k danému typu dizajnu) a *satisfaction_index* (celková spokojnosť).

Dáta z rôznych zdrojov boli zlúčené do jednej tabuľky na úrovni jednotlivých študentov, pričom boli odvodené aj ďalšie ukazovatele opisujúce výkon a správanie (napr. pomer interakcií, počet dodatočných pokusov, čas strávený v kvízoch).

Časové údaje boli upravené tak, aby minimalizovali vplyv neaktivity. Úlohy boli zároveň kategorizované podľa tém, čo umožnilo analyzovať výkonnosť naprieč rôznymi oblasťami učiva.

Limitácie experimentu

Experimentálne výsledky sú obmedzené viacerými faktormi. Dáta boli anonymizované, čo znemožnilo prepojenie správania študentov s ich postojmi. Skupiny mali nerovnakú veľkosť (PV: 21, CV: 14) a časť účastníkov vypadla, čo znižuje štatistickú silu výsledkov. Zároveň rozdielna počiatočná úroveň znalostí komplikuje porovnanie skupín.

Krátke trvanie kurzu (1 hodina) neumožnilo implementovať pokročilejšie didaktické prvky ani dlhodobéjšie učenie.

Absencia automatizovaného hodnotenia diagramov a nemožnosť zásahu učiteľa viedli k použitiu zjednodušených cvičení, čo mohlo ovplyvniť realizmus úloh a výsledky experimentu.

Zhrnutie experimentov

Výsledky ukazujú, že objektívne výkony boli vyššie v CV (Fig 8.3), zatiaľ čo miera zlepšenia bola vyššia v PV. Tento rozdiel je do veľkej miery vysvetlený vyššou počiatočnou úrovňou vedomostí v CV, čo obmedzovalo priestor na zlepšenie (Fig 8.2). Celkovo sa však nepreukázal jednoznačný rozdiel v celkových učebných výsledkoch medzi oboma prístupmi.

Z hľadiska správania študentov sa prejavili rozdielne vzorce interakcie. Študenti v CV častejšie opakovali kvízové úlohy a orientovali sa na dosahovanie merateľných výsledkov, zatiaľ čo študenti v PV viac pracovali s teoretickým obsahom, čo naznačuje viac exploratívny prístup (Fig 8.3).

Subjektívne hodnotenia ukazujú, že PV dosahuje vyššiu mieru spokojnosti, angažovanosti a celkového pozitívneho vnímania vo väčšine metrik (Fig 8.6).

Na druhej strane sa v PV pozoroval pokles záujmu o softvérové modelovanie po absolvovaní kurzu, čo môže naznačovať riziko oslabenia intrinzickej motivácie pri nevhodnom návrhu zábavných prvkov. Tento efekt však nebol štatisticky signifikantný (Fig 8.7).

CV vykazuje vyššiu variabilitu odpovedí, čo naznačuje polarizujúci efekt kompetitívneho dizajnu – časť študentov ho hodnotí veľmi pozitívne, iní menej priaznivo. PV je v tomto smere konzistentnejší a prístupnejší širšiemu spektru študentov (Fig 8.5).

Celkové výsledky naznačujú, že kompetitívny dizajn môže byť vhodnejší pre študentov s vyššou vstupnou úrovňou a záujmom o predmet, zatiaľ čo zábavný dizajn je všeobecne prístupnejší a vedie k vyššej subjektívnej spokojnosti.

Študenti zároveň prejavili záujem o ďalšie e-learningové materiály na FIIT STU, čo podporuje význam ďalšieho rozvoja e-learningu v oblasti softvérového modelovania (Fig 8.10).

Zhrnutie práce

Softvérové modelovanie a UML sú dôležitou súčasťou vzdelávania softvérových inžinierov. Napriek tomu študenti často preukazujú len povrchové alebo nepresné znalosti UML.

Problémy tradičného vyučovania, najmä nedostatok spätnej väzby a obmedzený individuálny prístup, môžu byť čiastočne riešené pomocou e-learningu. Ten umožňuje okamžitú spätnú väzbu, interaktívne úlohy a personalizované učenie, čo zvyšuje efektivitu štúdia.

Cieľom práce bolo vytvoriť a experimentálne overiť e-learningový kurz pre UML triedny diagram, pričom boli porovnané dva prístupy gamifikácie – zábavný a kompetitívny.

Výsledky ukázali rozdiely najmä v správaní a subjektívnom vnímaní študentov. Zábavný dizajn bol spojený s vyššou mierou spokojnosti, angažovanosti a záujmu o ďalší e-learning, zatiaľ čo kompetitívny dizajn vykazoval väčšiu variabilitu odpovedí a viac cieľovo orientované správanie. Rozdiely v objektívnych výsledkoch učenia neboli jednoznačné.

Celkové výsledky naznačujú, že e-learning má potenciál zlepšiť výučbu softvérového modelovania a jeho ďalší rozvoj na FIIT STU je opodstatnený.

Ďalšie smerovanie výskumu

Ďalší výskum by sa mal zamerať na rozšírenie kurzu o automatizované hodnotenie diagramov, generovanie cvičení a odporúčacie systémy. Perspektívne je aj skúmanie adaptívneho učenia a dlhodobějších experimentov, ktoré by umožnili presnejšie vyhodnotenie efektov jednotlivých prístupov. Do kurzu sa môžu pridať ďalšie druhy diagramov, prípadne sa dajú skúmať aj iné prístupy okrem zábavného a kompetitívneho.

List of Abbreviations

CV Competitive variant

FIIT STU Faculty of Informatics and Information Technologies STU in Bratislava

Gen AI Generative artificial intelligence

LMS Learning Management Systems

PV Playful variant

UML Unified Modeling Language

VPN Virtual private network

List of Figures

2.1	Hierarchy of UML diagrams	7
2.2	An example of a class	10
2.3	An example of a class diagram.	12
6.1	Activity diagram from a student's perspective.	37
6.2	Class diagram of the designed system.	42
6.3	Sequence diagram of the whole system's workflow.	43
7.1	Navigation of the theory section	46
7.2	Gradually building a diagram while discussing modeling challenges as they arise	47
7.3	Discussion of design details in the example diagram	47
7.4	Exercise for matching code with corresponding diagrams	49
7.5	Simulating modeling with the drag and drop onto picture activity	49
7.6	Picking correct improvement suggestions for a flawed diagram	50
7.7	Creating a custom report.	51
7.8	Example of a custom report.	51
7.9	A sample from feedback questionnaire for students.	52
7.10	Viewing the results of the feedback questionnaire.	53
7.11	Clear separation of topics for microlearning	53
7.12	A student's performance across various categories.	54
7.13	An incorrect answer displays a hint to the student.	54
7.14	Related theory section is suggested after an incorrect answer.	55
7.15	An exercise's difficulty is marked by a visual indicator	55
7.16	Solving an exercise correctly grants a reward	56
7.17	Badges were awarded for reaching certain milestones in the course . . .	56
7.18	Theory being explained in an informal, playful tone	57
7.19	10 best performing students are displayed in the leaderboard along with their scores	58

List of Figures

7.20	Students can compare their performance across various categories with the course's average	58
7.21	The course's average score is displayed when solving an exercise	59
7.22	Theory being explained in an assertive, demanding tone	59
8.1	Self-reported skill in UML before and after taking the course	68
8.2	Improvement vs score in the introductory assessment quiz	69
8.3	Average score by tag for Competitive and Playful variants	70
8.4	Improvement, final grade and intro score by time spent on the course . .	71
8.5	Distributions of subjective metrics in the feedback questionnaire	73
8.6	Difference of means (PV-CV) in engagement and general satisfaction metrics in the feedback questionnaire	74
8.7	Perceptions of student before and after taking the courses	75
8.8	Correlation matrix for feedback questionnaire - CV	76
8.9	Correlation matrix for feedback questionnaire - PV	76
8.10	Students' opinions on e-learning at FIIT STU	77
A.1	The first half of the structure of the ZIP file	B-2
A.2	The second half of the structure of the ZIP file	B-2
B.1	Restore a course from a backup in Moodle	B-4
B.2	Restore a backup as a new course	B-4
B.3	Installing a plugin from a ZIP file in Moodle	B-5
B.4	Recreating Configurable reports in Moodle, step 1	B-6
B.5	Recreating Configurable reports in Moodle, step 2	B-6
B.6	Recreating Configurable reports in Moodle, step 3	B-7
B.7	Recreating a Generico filter in Moodle	B-7

List of Tables

2.1	Comparison of UML tools: Enterprise Architect, Visual Paradigm, and Diagrams.net	8
3.1	Comparison of selected open-source Learning Management Systems . .	19
4.1	Summary of notable e-learning tools and methods for teaching UML and software modeling	26
8.1	Self-reported UML skill ratings across course variants before (Introduction) and after (Feedback) the intervention.	68
8.2	Descriptive statistics of performance metrics for competitive (C) and playful (P) course variants.	69
8.3	Simplified comparison of engagement behavior between Competitive (C) and Playful (P) variants.	71

References

- [1] Kleinner Farias and Bruno C da Silva. “What’s the grade of your diagram? towards a streamlined approach for grading UML diagrams”. In: *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. 2020, pp. 1–2.
- [2] Rosni Ramle et al. “UML Diagram E-Learning Application: A Pilot Study”. In: *Multidisciplinary Applied Research and Innovation* 2.1 (2021), pp. 418–425. doi: 10 . 30880/mari . 2021 . 02 . 01 . 044.
- [3] Mantas Jurgelaitis et al. “Implementing gamification in a university-level UML modeling course: A case study”. In: *Computer Applications in Engineering Education* 27.2 (2019), pp. 332–343. doi: 10 . 1002/cae . 22077.
- [4] Christian Cagnazzo et al. “Umlegend: A gamified learning tool for conceptual modeling with uml class diagrams”. In: *Proceedings of the 2nd International Workshop on Gamification in Software Development, Verification, and Validation*. 2023, pp. 2–5. doi: 10 . 1145/3617553 . 3617883.
- [5] Lvar Jacobson and James Rumbaugh Grady Booch. “The unified modeling language reference manual”. In: (2021). URL: <http://debracollege.dspaces.org/bitstream/123456789/404/1/UML%20Reference%20Manual%20by%20James%20Rumbaugh.pdf>.
- [6] Martina Seidl et al. *UML@ classroom*. Springer, 2015. doi: 10 . 1007/978-3-319-12742-2.
- [7] Bernhard Rumpe. *Modeling with UML*. Vol. 98. Springer, 2016. doi: 10 . 1007/978-3-319-33933-7.
- [8] Josep Soler et al. “A formative assessment tool for conceptual database design using UML class diagram”. In: *International Journal of Emerging Technologies in Learning (ijET)* 5.3 (2010), pp. 27–33. doi: 10 . 3991/ijet . v5i3 . 1402.
- [9] Iman Said Al-Darei and Abdelrahman Elhag. “The effect of feedback type in the e-learning environment on students’ achievement and motivation”. In: *Journal of Educational Technology and Online Learning* 5.3 (2022), pp. 694–705. doi: 10 . 31681/jetol . 1111527.

- [10] Yung-Ming Cheng. "What makes learners enhance learning outcomes in MOOCs? Exploring the roles of gamification and personalization". In: *Interactive Technology and Smart Education* 21.2 (Aug. 2023), pp. 308–330. ISSN: 1741-5659. DOI: 10.1108/ITSE-05-2023-0097. eprint: <https://www.emerald.com/itse/article-pdf/21/2/308/9581152/itse-05-2023-0097.pdf>. URL: <https://doi.org/10.1108/ITSE-05-2023-0097>.
- [11] Rui Huang et al. "The impact of gamification in educational settings on student learning outcomes: A meta-analysis". In: *Educational Technology Research and Development* 68.4 (2020), pp. 1875–1901. DOI: 10.1007/s11423-020-09807-z.
- [12] Zhanni Luo. "Gamification for educational purposes: What are the factors contributing to varied effectiveness?" In: *Education and Information Technologies* 27.1 (2022), pp. 891–915. DOI: 10.1007/S10639-021-10642-9.
- [13] WANG Chin-Yeh, KE Shu-Yu, et al. "E-learning system design with humor and empathy interaction by virtual human to improve students' learning". In: *International Conference on Computers in Education*. 2010.
- [14] Antonio Bucchiarone et al. "Gamifying model-based engineering: The PapyGame tool". In: *Science of Computer Programming* 230 (2023), p. 102974. ISSN: 0167-6423. DOI: <https://doi.org/10.1016/j.scico.2023.102974>. URL: <https://www.sciencedirect.com/science/article/pii/S0167642323000564>.
- [15] Sonja Trapp et al. "Collaborative learning of UML and SysML". In: (2011). DOI: 10.3991/ijep.v1i2.1663.
- [16] Rupinder Kaur, Muhammad Zubair Malik, and Maninder Singh. "Improving Delivery of UML Class Diagrams Concepts in Computer Science Education Through Collaborative Learning". In: *2023 IEEE Frontiers in Education Conference (FIE)*. IEEE. 2023, pp. 1–5.
- [17] Rebecca Reuter et al. "Insights in students' problems during UML modeling". In: *2020 IEEE Global engineering education conference (EDUCON)*. IEEE. 2020, pp. 592–600. DOI: 10.1109/EDUCON45650.2020.9125110.
- [18] Ronit Shmallo and Tammar Shrot. "Constructive use of errors in teaching the UML class diagram in an IS engineering course". In: *Journal of Information Systems Education* 31.4 (2020), pp. 282–293. URL: <http://jise.org/Volume31/n4/JISEv31n4p282.html>.
- [19] Stanislav Chren et al. *Mistakes in UML Diagrams: Analysis of Student Projects in a Software Engineering Course*. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. 100–109. 2019. DOI: 10.1109/ICSE-SEET.2019.00019.
- [20] Joachim Schramm et al. "Teaching UML Skills to Novice Programmers Using a Sample Solution Based Intelligent Tutoring System." In: *FLAIRS*. 2012. URL: <https://cdn.aaai.org/ocs/4388/4388-21578-1-PB.pdf>.

- [21] Gustav Bergström et al. "Evaluating the layout quality of UML class diagrams using machine learning". In: *Journal of Systems and Software* 192 (2022), p. 111413. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2022.111413>. URL: <https://www.sciencedirect.com/science/article/pii/S016412122200125X>.
- [22] Salisu Modi, Hanan Taher, and Hoger Mahmud. "A Tool to Automate Student UML diagram Evaluation". In: *Academic Journal of Nawroz University* 10 (Sept. 2021), pp. 189–198. DOI: 10.25007/ajnu.v10n2a1035.
- [23] Sara H. S. Almadi, Norhayati Mohd Ali, and Novia Admodisastro. "Class Diagram Recommender System (CDRS): An Educational Tool for UML Class Diagram". In: *Proceedings of the University Carnival on e-Learning (IUCEL)*. ID No. UPM 019. 2018, pp. 434–437.
- [24] Timothy Lethbridge et al. "Teaching UML using umple: Applying model-oriented programming in the classroom". In: May 2011, pp. 421–428. DOI: 10.1109/CSEET.2011.5876118.
- [25] Jung Tae Kim and Won-Hyung Lee. "Dynamical model for gamification of learning (DMGL)". In: *Multimedia Tools and Applications* 74.19 (2015), pp. 8483–8493. DOI: 10.1007/s11042-013-1612-8.

References

Declaration about the use of Generative Artificial Intelligence

OpenAI (2025), ChatGPT, Section 2.2, generation of Table 2.1

OpenAI (2025), ChatGPT, Section 2.3.2, text generation

OpenAI (2026), ChatGPT, Sections 2.3.3 and 6.3, consulting the design of diagrams

OpenAI (2026), ChatGPT, Section 3.4, text generation

OpenAI (2025), ChatGPT, Section, 4.1, generation of summaries in Table 4.1

OpenAI (2026), ChatGPT, Sections 7.1, help with creation of content displayed in courses
(generation of text, consulting exercise design)

OpenAI (2026), ChatGPT, Chapter 8, generation of code for data analysis, language
Python

OpenAI (2026), ChatGPT, Section 8.9, reformulation of text

OpenAI (2026), ChatGPT, Section 9.2 Future Work, reformulation of text

OpenAI (2026), ChatGPT, Chapter Résumé, summary generation, text translation

Appendix A

Description of Digital Submission

Thesis Evidence Number:

FIIT-182905-116308

Name of submitted archive:

DP_MatejSevcik.zip

The digital part included in the thesis contains the following files:

Appendix B

Technical Documentation

B.1 Installation Manual for Local Development

The installation guide assumes the use of Windows 10 or 11 operating systems.

1. Moodle provides installer packages for Windows that contain everything that is needed to run the application server locally. Download the latest stable release [from the official website](#) (the version name should not end with dev. E.g. - instead of downloading *Moodle 5.3dev*, download *Moodle 5.2*)
2. Create a directory in a location where you want your local Moodle instance to reside
3. Extract the installer package into the newly created directory
4. Navigate to the extracted directory and run `Start Moodle.exe`
5. Visit <http://localhost/> and follow displayed instructions to complete the installation
6. (Optional) If you want to recreate the courses used in this study:
 - (a) Log in as your admin user
 - (b) Install plugins according to instructions (Section B.3)
 - (c) Navigate to **Site administration** -> **Courses** -> **Restore course** ([Course restore documentation](#))
 - (d) Drop one of the attached backups (`course_backups/competitive_backup_no_users.mbz` or `course_backups/playful_backup_no_users.mbz`) into the drop zone and click **Restore**. Do so for both attached courses individually

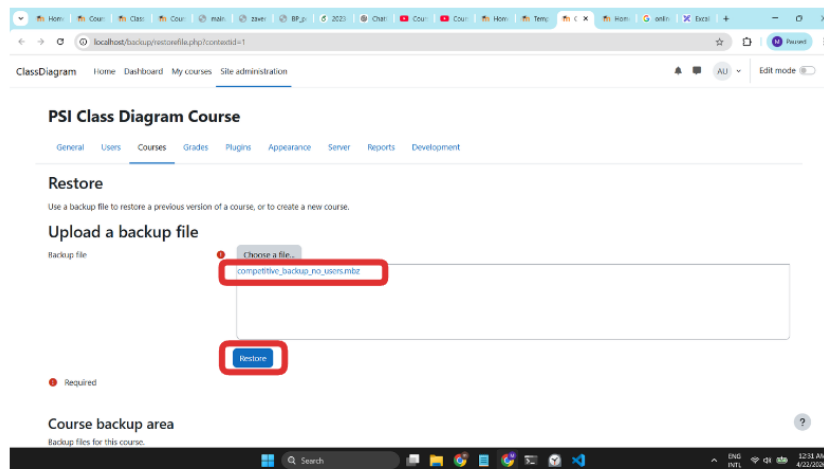


Figure B.1: Restore a course from a backup in Moodle

- (e) All the restore settings can be left at defaults. Pay care to select **Restore as new course** instead of **Restore into a course**

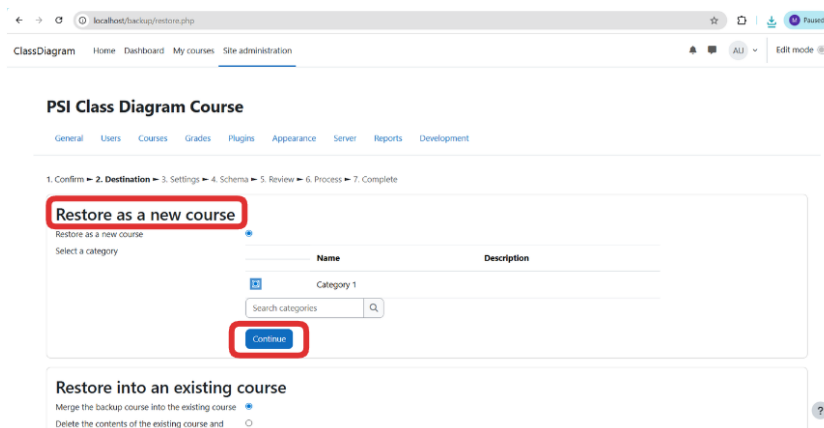


Figure B.2: Restore a backup as a new course

- (f) Once the restore is pending, you need to run the cron task. On a production server, it runs automatically, but locally it must be executed manually from the command line. Open a terminal and run (the entire text is one command):
- ```
"<path-to-moodle>\server\php\php.exe"
"<path-to-moodle>\server\moodle\admin\cli\cron.php"
```

## B.2 Installation Manual for Production Environment

1. Obtain access to a Linux server (Recommended: Ubuntu 24.04.4 LTS)
2. Set up the server according to [instructions from Moodle documentation](#)
3. (Optional) To recreate the courses used in this study, follow instructions from the previous section (item 6)

## B.3 Plugin Installation

Plugins are not restored using course backups. Courses from this study work without plugins, but some of the e-learning features are not available without them. They are used for getting live statistics and reports. To recreate them, follow these instructions - they are the same for both production and local environments:

1. Download plugins [Generico filter](#) and [Configurable reports](#)
2. Navigate to **Site administration -> Plugins -> Install plugins** ([Plugin installation documentation](#)). Drop the downloaded zip files into the displayed field and click **Install plugin from the ZIP file**

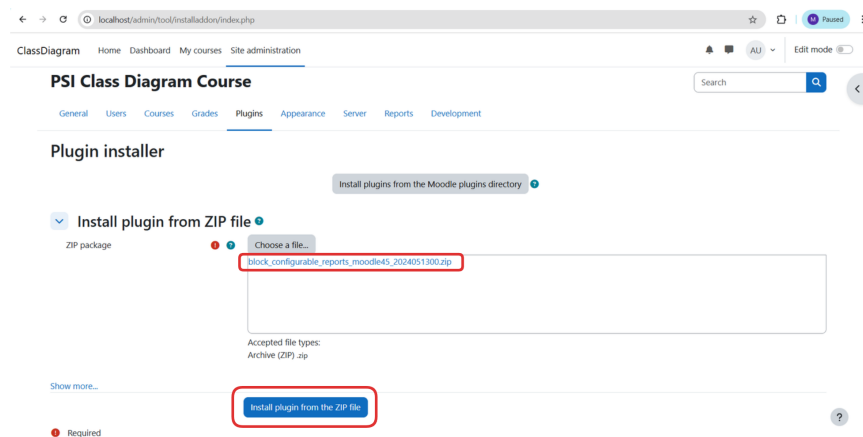


Figure B.3: Installing a plugin from a ZIP file in Moodle

### 3. Configurable reports

- (a) Navigate to <course-home> side panel and go to **Configurable Reports -> Manage reports**

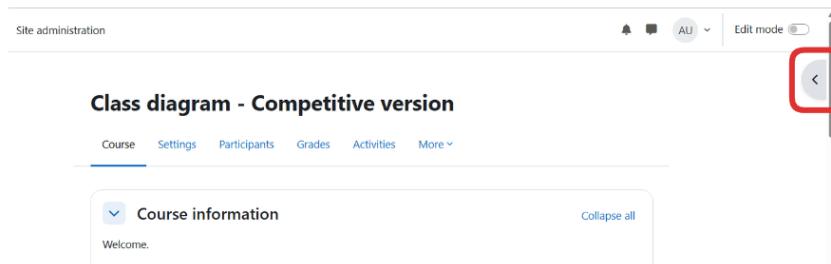


Figure B.4: Recreating Configurable reports in Moodle, step 1

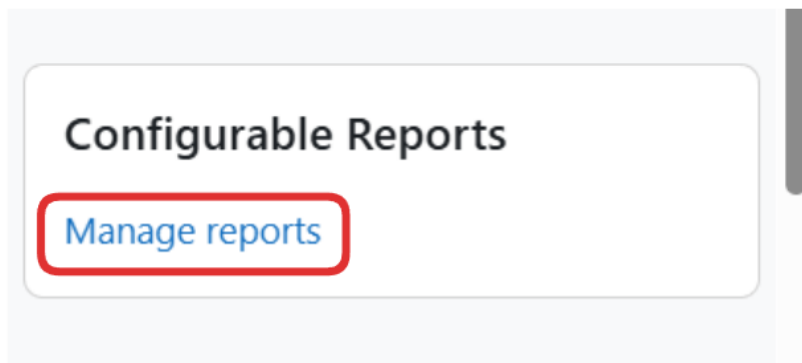


Figure B.5: Recreating Configurable reports in Moodle, step 2

- (b) Drag each report backup (`configurable_reports/competitive_course_backups/interactions.`) to the drop area in the corresponding course

## Appendix B. Technical Documentation

ClassDiagram Home Dashboard My courses Site administration

version

Add report

Import report from repository

Categories

Report

Import report

File

Choose a file...

Interactions.xml

Import report

Required

Figure B.6: Recreating Configurable reports in Moodle, step 3

### 4. Generico filter

- (a) Go to **Site administration -> Plugins -> Filters -> Category: Generico -> Templates** and click on one of the blank filters

Filters / Generico / Templates

PSI Class Diagram Course

General Users Courses Grades Plugins Appearance Server Reports Development

Generico Templates Admin

| Name         | Template key | Version | Description                                                                          |
|--------------|--------------|---------|--------------------------------------------------------------------------------------|
| Welcome User | welcomeuser  | 1.0.1   | This is a simple demonstration of how to display the current Users name and picture. |

2

3

4

Figure B.7: Recreating a Generico filter in Moodle

- (b) copy values from `generico/generico_question_avg.txt` to their respective fields
- (c) Each time the filter is used in the courses, the id in the filter is hardcoded. That means it needs to be manually updated in each instance (e.g. questions

## Appendix B. Technical Documentation

---

in the Competitive course contain {GENERIC0:type=question\_avg,qid=507}  
in their descriptions - the id will need to be updated)

# Appendix C

## Work Schedule

### C.1 Work Schedule for DP1

|            |                                                                 |
|------------|-----------------------------------------------------------------|
| week 1     | planning and outlining the work, researching the topic          |
| week 2-3   | writing about UML and the class diagram                         |
| week 4-5   | research and writing about e-learning                           |
| week 6-8   | researching the applications of e-learning to the class diagram |
| week 9-10  | writing about e-learning's applications to the class diagram    |
| week 11-12 | proposing our approach to the rest of the project               |

**Statement on the execution of the plan:** The purpose of DP1 was to research the topic of the project. At the beginning of the semester, exploration of UML and e-learning was carried out. We described the basic concepts of the class diagram. The subsequent topics were drawn out, however. We got delayed at research about e-learning and its applications on UML, and had to compensate for the lost time during the exam period.

## C.2 Work Schedule for DP2

|           |                                                          |
|-----------|----------------------------------------------------------|
| week 1    | research about various LMS                               |
| week 2-3  | familiarizing with Moodle LMS                            |
| week 4-6  | implementing the theoretical sections of the course      |
| week 7-11 | implementing the concepts of the exercises in the course |
| week 12   | writing the design and implementation chapters           |

**Statement on the execution of the plan:** The goal for DP2 was to implement the course. The intentions were not completely fulfilled, as we still have a number of tasks left - namely creating more exercises, and implementing e-learning techniques. We did create a draft of the course as well as drafts for all the exercise types we want to use.

### C.3 Work Schedule for DP3

|            |                                                        |
|------------|--------------------------------------------------------|
| week 1-2   | implementing e-learning techniques                     |
| week 3-4   | finishing the creation of additional exercises         |
| week 5     | deployment of the course on a private FIIT STU server  |
| week 6-10  | performing experiments on students and evaluating them |
| week 10-12 | finishing writing the thesis                           |

We finished the implementation of the course and deployed it on a FIIT STU server. The experiment started almost 2 weeks later than intended, but still allowed enough time for students to fill it in. Time for finishing the thesis was then reduced as well.